RV College of Engineering

Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

Mitigating Logistics Friction in Global Chokepoints: An Agentic AI Framework for Indian Export-Import Resilience

MAJOR PROJECT REPORT
(CD481P)

Submitted by

Anant Tewari	**1RV22CD005**
Kiran R Aithal	**1RV22CD022**
Pranav S K	**1RV22CD041**

Under the guidance of

Dr. Soumya A

Professor and Program Coordinator

Department of Computer Science and Engineering (Data Science)

RV College of Engineering

In partial fulfillment for the award of degree of

Bachelor of Engineering

in

Computer Science and Engineering

Department of Computer Science and Engineering

2025-26

RV College of Engineering®
Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

Go, change the world

Mitigating Logistics Friction in Global Chokepoints: An Agentic AI Framework for Indian Export-Import Resilience

A Major Project Report (CD481P)

Submitted by

Anant Tewari **1RV22CD005**

Kiran R Aithal **1RV22CD022**

Pranav S K **1RV22CD041**

Under the guidance of

Dr. Soumya A
Professor and Program Coordinator
Department of Computer Science and Engineering (Data Science)
RV College of Engineering

In partial fulfillment of the requirements for the degree of

Bachelor of Engineering in

Computer Science and Engineering

2025-26

RV College of Engineering, Bengaluru

(*Autonomous institution affiliated to VTU, Belagavi*)

Department of Computer Science and Engineering

CERTIFICATE

Certified that the major project (CD481P) work titled ***Mitigating Logistics Friction in Global Chokepoints: An Agentic AI Framework for Indian Export-Import Resilience*** is carried out by **Anant Tewari** (**1RV22CD005**), **Kiran R Aithal** (**1RV22CD022**) and **Pranav S K** (**1RV22CD041**) who are bonafide students of RV College of Engineering, Bengaluru, in partial fulfillment of the requirements for the degree of **Bachelor of Engineering** in **Computer Science and Engineering** of the Visvesvaraya Technological University, Belagavi during the year 2025-26. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the major project report deposited in the departmental library. The major project report has been approved as it satisfies the academic requirements in respect of major project work prescribed by the institution for the said degree.

Guide and Program Coordinator	Dean CS Cluster	Principal
Dr. Soumya A	Dr. Ramakanth Kumar P	Dr. K. N. Subramanya

External Viva

Name of Examiners	Signature with Date
1.	
2.	

DECLARATION

We, **Anant Tewari**, **Kiran R Aithal** and **Pranav S K** students of eighth semester B.E., Department of Computer Science and Engineering, RV College of Engineering, Bengaluru, hereby declare that the major project titled '**Mitigating Logistics Friction in Global Chokepoints: An Agentic AI Framework for Indian Export-Import Resilience**' has been carried out by us and submitted in partial fulfilment for the award of degree of **Bachelor of Engineering** in **Computer Science and Engineering** during the year 2025-26.

Further we declare that the content of the dissertation has not been submitted previously by anybody for the award of any degree or diploma to any other university.

We also declare that any Intellectual Property Rights generated out of this project carried out at RVCE will be the property of RV College of Engineering, Bengaluru and We will be one of the authors of the same.

Place: Bengaluru

Date:

Name	Signature
1. Anant Tewari(1RV22CD005)	
2. Kiran R Aithal(1RV22CD022)	
3. Pranav S K(1RV22CD041)	

ACKNOWLEDGEMENTS

We are indebted to our guide, **Dr. Soumya A**, Professor and Program Coordinator, Department of CSE, RVCE for the wholehearted support, suggestions and invaluable advice throughout our project work and also helped in the preparation of this thesis.

Our sincere thanks to the project coordinators **Dr. Sindhu D V**, Assistant Professor and Program Coordinator, Department of Computer Science and Engineering (Data Science) and **Dr. Chethana Murthy**, Associate Professor and Program Coordinator, Department of Computer Science and Engineering for their timely instructions and support in coordinating the project.

Our sincere thanks to **Dr. Ramakanth Kumar P**, Professor and Dean CS Cluster, Department of Computer Science and Engineering, RVCE for the support and encouragement.

We express sincere gratitude to our beloved Professor and Vice Principal, **Dr. Geetha K S**, RVCE and Principal, **Dr. K. N. Subramanya**, RVCE for the appreciation towards this project work.

We thank all the teaching staff and technical staff of Computer Science and Engineering department, RVCE for their help.

Lastly, we take this opportunity to thank our family members and friends who provided all the backup support throughout the project work.

ABSTRACT

Global supply chains are increasingly susceptible to volatility, with geopolitical tensions, climate events, and infrastructure bottlenecks creating significant logistics friction. For Small and Medium Enterprises (SMEs) in India, these disruptions are particularly damaging due to a reliance on manual reactive planning and a lack of real-time visibility. Traditional logistics management often fails to account for cascading delays, leading to inflated safety stocks, high landed costs, and missed service-level agreements (SLAs). The core problem addressed in this work is the inability of current SME logistics frameworks to autonomously detect and mitigate these frictions in an efficient, data-driven manner.

This research proposes an *Agentic Supply-Chain Digital Twin* designed to enhance resilience through autonomous orchestration. The solution integrates a directed-graph model representing the physical logistics topology with a Multi-Agent System (MAS) built on the CrewAI framework. The system features a "Friction Sentry" that detects disruption signals and maps them to severity multipliers, a mathematical digital twin that computes lead-time impacts, and LLM-powered agents that autonomously evaluate alternate routing corridors. By combining deterministic graph algorithms with agentic reasoning, the framework provides actionable mitigation strategies—such as dynamic rerouting and emergency replenishment—that significantly reduce human intervention in the decision-making loop.

The proposed system was evaluated against a human-led baseline across a suite of ten deterministic supply chain shock scenarios including port congestions and maritime obstructions. Evaluation metrics demonstrate that the agentic pipeline achieves a significant response speedup, reducing identification-to-resolution latency by an average of 11.95 hours. Furthermore, the system successfully identified optimized alternate paths that reduced total landed costs by an average of $1,927.58 per event while maintaining inventory cover above critical safety thresholds. These results validate the effectiveness of the autonomous digital twin in providing a scalable, resilient, and cost-effective approach to modernizing SME logistics management.

CONTENTS

LIST OF FIGURES

LIST OF TABLES

ABBREVIATIONS

DT Digital Twin

LLM Large Language Model

MAS Multi-Agent System

SME Small and Medium Enterprise

TEU Twenty-foot Equivalent Unit

WTO World Trade Organization

Chapter 1

Introduction

CHAPTER 1

INTRODUCTION

A comprehensive introduction to the proposed project is provided here, outlining the background of the problem domain, the significance of the work, the objectives, the scope, and the methodology adopted. Research gaps are identified based on a review of existing literature, and the overall organisation of the report is outlined. The purpose of this introduction is to provide readers with a clear understanding of the motivation, need, and approach of the proposed work.

1.1 Introduction

Global trade is increasingly defined by *Logistics Friction*—a composite of temporal, financial, and physical delays at critical maritime chokepoints. Geopolitical tensions, extreme weather events, and infrastructure bottlenecks have been shown to amplify disruptions across interconnected supply chains, with cascading consequences for economies heavily dependent on maritime trade. Specifically, the supply and logistics of Crude Oil, LNG, and other petroleum products and by-products have been demonstrated to be particularly vulnerable to such disruptions.

While large-scale enterprises employ sophisticated risk-modelling tools, Indian Small and Medium Enterprises (SMEs) remain exposed to the *Bullwhip Effect*—a supply chain phenomenon wherein small fluctuations in consumer demand at the retail level cause progressively larger fluctuations upstream, at the wholesale, distributor, and manufacturer levels. This vulnerability is compounded by sudden shifts in transit verification procedures, insurance premium spikes, and port congestion, all of which can render manual planning processes wholly inadequate.

The emergence of Agentic Artificial Intelligence (AI) and Large Language Model (LLM)-based multi-agent frameworks presents a transformative opportunity to bridge this gap. By enabling autonomous detection, analysis, and execution of logistics mitigation strategies, these technologies can equip Indian SMEs with capabilities previously reserved for multinational corporations. This project, therefore, proposes a novel *Agentic Supply-Chain Digital Twin*—an integrated framework capable of detecting disruption signals, quantifying their impact on supply networks, and autonomously executing optimised routing and inventory management strategies to protect Indian EXIM resilience.

The domain of this project resides at the intersection of Agentic Artificial Intelligence and Maritime Logistics, a sector critical to global trade. In modern systems, the ability to automate complex decision-making through autonomous agents has become a cornerstone of supply chain resilience. Recent advancements in Large Language Model (LLM)s and multi-agent systems have moved the field beyond static predictive models toward dynamic, goal-oriented frameworks capable of independent reasoning. This evolution holds immense academic and industrial relevance, as it directly addresses the persistent challenge of logistics friction within highly volatile maritime chokepoints.

By integrating real-time data ingestion with agent-driven optimisation, the proposed technology plays a vital role in mitigating the Bullwhip Effect and reducing transit uncertainties. For Indian enterprises, particularly Small and Medium Enterprise (SME)s, this framework offers a robust mechanism for maintaining export-import competitiveness, transforming reactive crisis management into proactive, intelligent resilience against global trade shocks. The project thus contributes to the growing body of work demonstrating that agentic AI is no longer merely a conceptual framework, but a practical design pattern for autonomous logistics governance.

1.1.1 Terminology and Definitions

This section defines the key technical terms, abbreviations, and domain-specific concepts used throughout this report.

- **Shock Location / Disruption Node**: A specific geographical point or transit corridor (e.g., chokepoint, port, road link) experiencing supply chain friction, congestion, or security threats.
- **Signal Type**: The classification of the disruption or external risk factor triggering a supply-chain alert.
- **Severity Multiplier**: A coefficient that scales baseline time and cost variables to simulate the intensity of a disruption.
- **Strategic Preference**: The trade-off priority chosen by a logistics manager (or agent) to optimise the supply network.
- **Landed Cost**: The total cumulative cost of buying, transporting, insuring, and storing inventory until it reaches its final destination market.
- **Transit Days**: The lead time (in days) required for cargo to travel along the chosen

path from the supplier to the destination market.

- **Response Latency**: The decision-making time elapsed between detecting a supply chain shock and implementing a mitigation strategy (such as rerouting or restocking).
- **Twenty-foot Equivalent Unit (TEU)**: A standardised unit of measurement for container capacity in maritime shipping, equivalent to one 20-foot container. Used to measure vessel capacity and cargo volume.
- **Bunker Cost**: The cost of marine fuel (bunker oil) required to power a vessel during transit. Bunker costs fluctuate based on global oil prices and significantly impact overall shipping expenses.
- **SME**: Small and Medium Enterprises—businesses that maintain revenues, assets, or a number of employees below certain thresholds. In this report, the term refers specifically to Indian firms engaged in export-import trade.
- **Multi-Agent System (MAS)**: Multi-Agent System—a system composed of multiple interacting intelligent agents that cooperate to solve complex problems.
- **LLM**: Large Language Model—a deep learning model trained on vast textual corpora to understand and generate human language.
- **Digital Twin (DT)**: Digital Twin—a virtual, real-time representation of a physical system used for simulation, monitoring, and decision support.

1.2 Literature Review

This section provides a detailed review of existing research, systems, technologies, and methodologies related to the project domain.

1.2.1 Recent Works

A body of recent literature has positioned agentic AI as a transformative force in supply chain management, shifting the focus from passive prediction to autonomous execution. The integration of real-time inventory management with agentic systems using distributed cloud architectures has been demonstrated, showing that forecasting can be effectively combined with operational action [1]. In retail contexts, agentic AI has been applied for predictive analytics, autonomous customer interaction, and supply chain automation, particularly where responsiveness and personalisation are essential [2], [3].

Autonomous decision-making has also been extended to food supply chains, where agentic frameworks are proposed as mechanisms for improving control in complex, time-sensitive environments [4].

These contributions are aligned with broader reviews of agentic AI that emphasise autonomy, adaptability, tool use, and multi-step goal pursuit as defining paradigm properties [5], [6]. It is thus established in the literature that agentic AI has moved beyond conceptual framing to serve as a practical design pattern for forecasting, interaction, and operational decision support in supply chains.

The technical foundations for autonomous supply chains have been established through multi-agent systems (MAS) and digital twin research. A systematic review of agent-based modelling and simulation has confirmed that MAS has long been applied to the study of decentralised coordination, negotiation, and disruption handling in supply chains [7]. Building on this foundation, autonomous supply chains have been implemented through multi-agent architectures that support monitoring, planning, and negotiation across supply chain tiers [8]. A digital-twin perspective has further argued that autonomous supply chains are strengthened by coupling agents with a simulation layer, enabling real-time scenario testing and adaptation [9].

The importance of coordination theory is central in this line of work, as agentic and multi-agent systems are fundamentally coordination mechanisms operating under uncertainty [10].

The scope of supply chain analytics has been significantly expanded by the recent literature on Large Language Models (LLMs), extending it from structured forecasting to unstructured risk interpretation and prescriptive response. A systematic review of LLMs in supply chain management has identified four major application domains: optimisation, risk and security management, knowledge management, and automated contract intelligence [11]. Complementing this synthesis, studies on hyperconnected logistic hubs and crisis response have shown that LLMs can support risk assessment, zero-shot reasoning, and prompt-based logistical reasoning under stress [12].

The most important implication established by this strand of research is that LLMs can help convert event streams, news, and operational text into actionable supply chain intelligence. This is especially evident in work on automating supply chain disruption monitoring through an agentic AI framework, where specialised agents have been shown

to detect events, map impacts across tiers, assess risks, and recommend responses [13]. The result is a demonstrated transition from descriptive reporting toward near-real-time prescriptive action. In this context, LLMs are not treated as isolated chat interfaces; they are embedded within agentic pipelines that connect language understanding with structured decision workflows [6], [13].

A parallel line of research has focused on how disruptions propagate through multi-tier networks. Graph-based Monte Carlo methods have been proposed to analyse cascading supply chain shocks across multiple tiers, and graph neural networks have been applied to map supply chain structure and model disruption propagation [14]. These methods are important because they make network interdependence explicit, which is essential in volatile environments where a local shock can become a system-wide failure.

The need for such modelling is further supported by earlier work on the ripple effect, which formalised how disturbances amplify across supply chains and connected this phenomenon to robustness, resilience, and viability [15], [16]. Related studies have demonstrated how digital technology and Industry 4.0 reshape ripple dynamics and supply chain risk analytics [17]. However, the literature has also emphasised that predictive power alone is insufficient. Machine-learning-based risk assessment has been shown to trade off performance against interpretability, which creates a governance problem in high-stakes decision contexts [18].

The sustainability literature shows that agentic and digital systems are increasingly being framed as instruments for aligning efficiency with environmental and social objectives. Agentic AI and ambient intelligence have been proposed together for autonomous sustainability decision-making in supply chains [19]. A related framework, SustAI-SCM, has positioned agentic intelligence as a mechanism for automating procurement, logistics, and inventory decisions while explicitly optimising cost and emissions [20]. These studies indicate that agentic supply chain design is expanding beyond speed and resilience toward environmental performance and long-horizon optimisation.

Enterprise systems are also being reconsidered through this lens. Agentic AI has been described as a force for ERP automation in supply chain management, where cognitive agents are proposed to replace static workflows with autonomous reasoning and execution [21]. In planning contexts, agentic AI is presented as a transition from optimisation to autonomous action, shifting the planner's role from system operator to strategic supervisor

[22].

The more established literature provides the conceptual and technological substrate for the newer agentic work. Digital twins are a central reference point: they have been applied to manage disruption risk in Industry 4.0 settings [23], and subsequent work has framed digital supply chain twins as a key direction for resilience research [24]. Reviews of digital twins in supply chains have emphasised visibility, agility, decision support, and integration with IoT, AI, and blockchain [24]. In parallel, the IoT literature has highlighted both the operational promise and the implementation risks of connected sensing in supply chain management [25], [26].

The resilience literature further demonstrates why these technologies matter. Supply chain viability under COVID-19, epidemic outbreaks, and shortage conditions has been extensively theorised [27], [28], [29]. Sector-specific studies on food consumption, humanitarian supply chains, and the automobile and airline industries have illustrated how disruptions alter demand, service delivery, and collaboration patterns [30], [31], [32]. The ripple effect remains a core analytical concept in this body of work and is repeatedly linked to robustness and adaptation strategies [33], [34].

Industry 4.0 and Industry 5.0 research has added another layer by showing how digital adoption, cyber-physical integration, and human-centricity reshape supply chain design [35], [36], [37]. Big data analytics and predictive analytics are treated as enabling capabilities for agility and performance measurement [38], [39], [40], [41], [42], [43]. Blockchain has also appeared as complementary infrastructure for transparency, trust, sustainability, and resilience [44], [45], [46], [47], [48], [49].

Across these streams, the literature has moved from predictive analytics and digital visibility toward agentic autonomy, multi-agent coordination, and prescriptive action. The strongest recent contributions show that autonomous systems can monitor disruptions, reason over text and network structure, and support sustainability-oriented decisions [11], [13], [19], [20]. However, the established literature also demonstrates that most existing work is either sector-specific, simulation-oriented, or focused on enabling technologies rather than end-to-end execution [1], [2], [8], [50].

The most important unresolved gap identified is the absence of a unified framework that combines agentic reasoning, disruption intelligence, explainability, and one-click operational execution for resource-constrained and volatility-prone contexts. This gap is

especially visible in maritime and SME-oriented environments, where disruption originates deep in the network, data are sparse, compliance is heavy, and response latency is critical [51], [52], [53], [54]. The present project therefore sits at the intersection of agentic AI, resilience analytics, and autonomous supply chain governance, with the primary goal of moving from detection to mitigation in a single operational loop.

Recent advancements in Large Language Models and multi-agent frameworks (such as CrewAI) have presented opportunities to make supply chain digital twins proactive. In agentic AI systems, specific roles (e.g., monitoring news, optimising routes, analysing costs, and generating executive reports) are assigned to specialised software agents. These agents are then equipped with tools to query the digital twin's database, run mathematical optimisation engines, and draft human-readable recommendations. This hybrid approach—combining the symbolic reasoning of graphs and pathfinding with the linguistic capability of LLMs—has been shown to bridge the gap between complex analytical calculations and operational decision-making.

1.2.2 Research Gap Identification

Based on the review of recent research papers, the following key research gaps have been identified:

- The automated translation of unstructured news alerts (e.g., geopolitical reports) into quantitative logistics penalties has not been adequately addressed in existing systems.
- Effective coordination between route optimisation (symbolic algorithms) and strategic inventory management (safety stock, restocking) is absent in current frameworks.
- A need for human-in-the-loop dashboard triggers that present visual recommendations alongside financial and temporal trade-offs has been identified but not fulfilled in the existing literature.

1.3 Motivation

The primary purpose of this project is to bridge the critical gap between disruption detection and autonomous response in maritime logistics. As global trade becomes increasingly volatile, the practical need for a system that can not only predict shocks but also execute mitigation strategies independently has become paramount. This work is

inspired by the necessity to protect vulnerable economic actors, such as Indian SMEs, from the cascading effects of geopolitical instability.

The motivation for this work stems from several fundamental challenges identified in existing systems:

- **Neglect of Upstream Maritime Shocks:** Existing predictive systems have been found to focus predominantly on downstream retail inventory and e-commerce environments, failing to account for critical upstream disruptions at maritime chokepoints such as the Suez Canal or the Red Sea.
- **The Detection-to-Action Gap:** Current multi-agent frameworks have been observed to excel at producing analytical reports from global signals, yet still require manual human execution to implement responses, leading to costly delays.
- **Reliance on Historical Data:** Modern replenishment models are known to depend heavily on historical datasets, leaving them unable to simulate or mitigate unpredictable, real-time geopolitical transit shocks that have no historical precedent.
- **Underutilisation of Agentic Tool Coordination:** While autonomous supply chain models have been proposed in the literature, they have often been found to fail in leveraging the full power of Agentic AI to coordinate diverse toolkits for the automated execution of complex user tasks, such as simultaneous rerouting and inventory restocking.

By addressing these limitations, this project holds immense industrial and societal importance, transforming traditional reactive supply chain management into a proactive, agent-driven digital twin capable of ensuring global trade resilience.

1.4 Problem Statement

Indian Small and Medium Enterprises engaged in global export-import operations are highly susceptible to maritime supply chain disruptions arising from geopolitical tensions, port congestion, and chokepoint blockades. In the absence of automated, real-time mitigation systems, these enterprises resort to manual, reactive logistics management, which is characterised by significant response latency, inflated safety stock requirements, and escalating landed costs. The prevailing gap between disruption detection and autonomous execution is therefore identified as the primary impediment to SME logistics resilience.

This project proposes an Agentic AI framework to bridge this gap by autonomously detecting, quantifying, and mitigating logistics friction in data-scarce, volatility-prone Indian EXIM environments.

1.4.1 Global Scenario

Technological developments in global trade are increasingly pivoting toward AI-driven resilience. Organisations such as the World Trade Organization (World Trade Organization (WTO)) and major shipping conglomerates are prioritising logistics friction mitigation to stabilise global maritime trade. Recent innovations include predictive risk-modelling and decentralised logistics orchestration, yet the worldwide demand for autonomous execution frameworks remains high. Global supply chains, particularly those passing through maritime chokepoints such as the Red Sea or the Suez Canal, face escalating vulnerabilities to geopolitical and administrative shocks.

While advanced economies are integrating multi-agent systems to handle these disruptions, a significant technological gap persists in transit corridors affecting developing markets. The maritime industry is currently shifting from simple problem detection to agent-driven real-time optimisation, with international research trends focusing on goal-oriented autonomous frameworks. This global demand highlights the significance of projects that create scalable, intelligent solutions for international trade stability, particularly for economies such as India, where EXIM resilience is intrinsically linked to the prosperity of the SME sector.

1.4.2 Societal Relevance

This project carries significant societal relevance by safeguarding the livelihoods dependent on India's export-import sector. Small and Medium Enterprises form the backbone of the Indian economy, yet they are disproportionately impacted by the Bullwhip Effect when maritime disruptions occur. By providing these enterprises with accessible, agentic AI tools, the project promotes economic equity and industrial sustainability. It transforms complex, manual logistics processes into automated, efficient workflows, reducing the financial strain caused by sudden insurance spikes or port congestion.

Furthermore, the framework supports national economic resilience by ensuring the steady flow of essential commodities such as crude oil and LNG. Societally, this leads to price stabilisation and increased security within manufacturing and trading commu-

nities. By empowering smaller industrial players with technology previously accessible only to large corporations, the project contributes to a more robust and equitable national economy, aligning with the broader objectives of India's Digital India and Atmanirbhar Bharat initiatives.

1.4.3 Problem Area

The core problem area is the critical gap between detection and autonomous execution in current logistics management systems. Existing maritime systems are predominantly reactive and manual, leaving Indian SMEs vulnerable to rapid disruptions at maritime chokepoints. When a transit delay occurs, these organisations often lack the data infrastructure and risk-modelling capabilities required to implement proactive mitigation strategies. Furthermore, most modern AI solutions are designed for data-rich environments, creating a data-infrastructure gap for SMEs operating in data-sparse, developing-economy contexts.

If left unaddressed, minor transit delays escalate into severe domestic supply chain failures and unmanageable landed costs. This problem requires immediate attention because traditional manual intervention is insufficient to manage the cascading effects of modern geopolitical shocks. An intelligent framework capable of independent reasoning and rapid response is therefore necessary to mitigate logistics friction and protect Indian EXIM operations from irreversible disruption.

1.5 Objectives

The objectives of this project, formulated to directly address the identified problem statement, are as follows:

(i) To design and implement a Multi-Agent System (MAS) using CrewAI that autonomously ingests unstructured geopolitical signals and quantifies them into measurable Logistics Friction variables within maritime chokepoint corridors, thereby eliminating the latency inherent in manual disruption detection.

(ii) To construct and validate a Logistics Digital Twin by mapping a multi-tier maritime and inland supply network using NetworkX, enabling the visualisation and quantification of shock propagation from international nodes to domestic Indian SME warehouses.

(iii) To enable autonomous one-click mitigation execution by deploying AI agents

capable of generating actionable recommendations—including alternate maritime routing, dynamic buffer-stock adjustments, and emergency restocking—that transition the system from analytical reporting to executable action.

(iv) To validate the framework's economic and operational efficacy by establishing a rigorous mathematical baseline and demonstrating significant, measurable reductions in landed cost and response latency compared to traditional, manual SME logistics planning processes.

1.6 Assumptions made / Constraints of the project

This section provides a detailed technical breakdown of the proposed Agentic AI framework, encompassing the core concepts, technological stack, and architectural design principles.

1.6.1 Technical Concepts

The project integrates several advanced domains to create a robust supply-chain solution:

- **Digital Twin:** A virtual, real-time representation of the physical supply-chain network—covering suppliers, ports, chokepoints, and warehouses—is used to simulate shocks and test responses.
- **Agentic Orchestration:** Multi-agent workflows are employed wherein specialised LLM-based autonomous agents collaborate via role-playing and prompt chaining to solve complex routing problems.
- **Graph Theory & Network Optimisation:** Logistics lanes are modelled as directed graphs and **Dijkstra's Algorithm** is applied to calculate optimal paths based on dynamic weights such as transit days and landed costs.
- **Stochastic Inventory Management:** The **Chopra Safety Stock Equation** is implemented to model lead-time variability and calculate buffer stocks required to avoid stockouts during shocks.

1.6.2 Technologies, Tools, and Frameworks

The implementation leverages a modern, high-performance technology stack:

- **Backend:** Python (FastAPI) for high-performance REST APIs and asynchronous processing.

- **Orchestration Framework: CrewAI** for agent role definition and task delegation, powered by **Gemini-Flash** LLMs via the Google AI Studio API.
- **Graph Modelling:** The `networkx` library for building and managing the directed graph of the global logistics network.
- **Frontend:** React (Vite) for a glassmorphism dashboard, using **Leaflet** (`react-leaflet`) for interactive, geospatial visualisation of routing paths.

1.6.3 System Architecture and Working Principle

The system operates as a reactive closed-loop pipeline where external shock events trigger a multi-stage response. First, the Friction Sentry maps unstructured signals to quantitative shock parameters. Second, the Agentic Optimiser uses specialised agents (Logistics Optimiser and Cost Analyst) to identify alternate routes and check inventory levels. Finally, the Communicator Agent compiles an actionable JSON payload to update the React frontend dashboard with optimised routing and financial metrics.

1.6.4 Real-World Applications

Practical applications of this framework include dynamic rerouting of cargo to avoid high-risk maritime zones (e.g., bypassing the Red Sea via the Cape of Good Hope) and port congestion mitigation by automatically switching destination ports (e.g., from Nhava Sheva to Mundra). Most significantly, Indian SMEs are empowered with automated inventory protection strategies that are typically only accessible to large multinational corporations.

1.7 Scope and Relevance

The scope of this project is concentrated on the development of an agentic digital twin for maritime logistics, specifically addressing upstream disruptions at global chokepoints. The scope includes real-time disruption parsing (Friction Sentry), autonomous multi-agent optimisation using CrewAI, and geospatial visualisation for Indian EXIM routes. The system incorporates dynamic inventory management using the Chopra equation and Dijkstra-based rerouting. The project scope excludes the orchestration of physical last-mile delivery fleets and does not integrate real-time satellite imagery for hardware-level port monitoring, relying instead on structured semantic signals.

The project holds high industrial relevance by offering SMEs technological tools pre-

viously reserved for multinational corporations, reducing the financial impact of the Bullwhip Effect. Academically, it contributes to the evolving field of agentic AI by demonstrating goal-oriented tool coordination for complex problem-solving. Societally, it fosters economic resilience through stabilised export-import flows, ensuring that vital supply chains remain operational despite geopolitical shocks.

1.8 Brief Methodology of the project

The methodology for this project is structured into five distinct phases, transitioning from network modelling to autonomous execution and validation.

- **Stage 1: Network Model Construction & Baseline Development:** A multi-tier logistics graph is developed using `NetworkX`, mapping critical nodes from international maritime chokepoints (e.g., Mundra, Dubai) to inland Indian SME clusters. A Manual Human Planner baseline is simultaneously established using Chopra inventory formulas to provide a performance benchmark for traditional, reactive supply chain management.
- **Stage 2: Multi-Agent Orchestration (Intelligence Layer):** Specialised autonomous agents are designed and deployed using `LangChain` and `CrewAI`. This includes a Friction Sentry for real-time monitoring of maritime signals (e.g., verification delays, insurance spikes, or congestion) and a Logistics Optimiser for executing autonomous rerouting and dynamic buffer-stock calculations.
- **Stage 3: Stress-Testing & Friction Propagation:** One simulated Friction Shock—ranging from administrative bottlenecks to regional blockades—is injected into the graph model. The agents identify these disruption signals, propagate the impact across the multi-tier supply chain, and evaluate the mathematical trade-offs of various mitigation strategies.
- **Stage 4: Autonomous Mitigation & Execution:** The agentic pipeline autonomously selects the most efficient recovery path, such as pivoting to multimodal transport or identifying backup suppliers, to minimise Total Landed Cost and operational downtime. This phase transitions the system from simple detection to executable action.
- **Stage 5: Visualisation & Performance Validation:** The agentic output is integrated into an interactive Dashboard for real-time decision support. The frame-

work's effectiveness is validated by comparing its response latency and cost-efficiency against the previously established manual baseline, targeting a measurable reduction in stockouts and overstocking.

1.9 Organisation of the Report

The remainder of this report is organised as follows:

- **Chapter 2 – Theory and Fundamentals:** The theoretical foundations of Multi-Agent Systems, the Chopra inventory model, and the graph-theoretic principles used in the Logistics Digital Twin are discussed.
- **Chapter 3 – Software Requirement Specification:** The detailed system requirements, functional specifications, and hardware/software prerequisites are defined.
- **Chapter 4 – High-Level Design:** The architectural design, agent roles, and the integration of CrewAI with the NetworkX graph model are elaborated.
- **Chapter 5 – Detailed Design:** The internal functional design, interaction protocols, and specific data structures used within the modular digital twin framework are focused upon.
- **Chapter 6 – Implementation Details:** The coding standards, module connectivity, and the end-to-end integration of the backend services with the interactive dashboard are documented.
- **Chapter 7 – Software Testing:** The verification of modules and system workflows through unit, integration, and performance benchmarking suites is presented.
- **Chapter 8 – Results and Discussion:** The system's performance metrics are analysed and the agentic optimisation results are validated against human benchmarks.
- **Chapter 9 – Conclusion and Future Scope:** The project's contributions are summarised, current limitations are identified, and a roadmap for future technical enhancements is proposed.

1.10 Summary

This chapter introduced the project domain, problem statement, objectives, and the significance of the proposed work. The background of maritime logistics disruption, a

comprehensive literature survey highlighting research gaps, the proposed methodology, and the expected outcomes of the project were discussed. The information presented in this chapter establishes the foundation for the detailed technical discussions provided in the subsequent chapters.

Chapter 2

Theoretical Foundations and Core Concepts

CHAPTER 2

THEORETICAL FOUNDATIONS AND CORE CONCEPTS

The academic and technical foundation for the proposed agentic AI framework is established in this section. The stochastic nature of inventory management under uncertainty is explored, along with the mathematical modelling of logistics networks through graph theory and the mechanics of disruption propagation. Furthermore, the role of multi-agent systems in autonomous orchestration and the conceptual shift towards strategic digital twins for proactive risk management are detailed.

2.1 Stochastic Inventory Control (Chopra & Meindl Framework)

In classical supply chain theory, inventory management must account for uncertainty in both demand and supply lead times. This project implements this using the **Chopra Inventory Model**:

- **Safety Stock (SS) Equation:**

$$SS = z \cdot \sqrt{\bar{L} \cdot \sigma_D^2 + \bar{D}^2 \cdot \sigma_L^2} \tag{2.1}$$

  where:

  - $\bar{L}$ (Average Lead Time): The shocked transit time is calculated dynamically by Dijkstra's algorithm.
  - σ_L (Lead Time Deviation): This is represented as $\sigma_L = \bar{L} \cdot (\text{severity} - 1) \cdot 1.5$, showing how disruption severity directly inflates lead-time volatility.
  - z (Service Level Factor): The inverse cumulative distribution function of the standard normal distribution (Z-score). This project locks this at 1.645 for a 95% service level.

- **The Concept:** Traditional systems assume static lead times. This theory proves that as lead time ($\bar{L}$) and lead time uncertainty (σ_L) grow due to disruptions, the required safety stock increases exponentially to maintain service levels, directly driving up holding costs.

2.2 Dynamic Network Flow & Dijkstra's Algorithm

The physical transport network is represented mathematically as a **Weighted Directed Graph** $G = (V, E)$, where V represents nodes (Suppliers, Ports, Warehouses, Markets) and E represents directed routes.

- **Edge Weight Dynamism:** Unlike static routing systems, edge weights are dynamic variables:

$$w_e = f(\text{baseline\_weight}, \text{shock\_multiplier}, \text{semantic\_rules}) \tag{2.2}$$

- **Dijkstra's Shortest Path Algorithm:** The routing engine is used to solve the single-source shortest path problem to find a path P from Supplier S to Market M that minimises:
  - $\sum_{e \in P} \text{transit\_time}_e$ (when minimising Time)
  - $\sum_{e \in P} \text{freight\_cost}_e$ (when minimising Cost)
- **The Concept:** Local disruptions (e.g., blockades in the Strait of Hormuz) alter global network paths, shifting bottleneck dynamics to other nodes (like Mundra or Nhava Sheva ports).

2.3 Supply Chain Disruption & Cost of Inaction (CoI)

A comparative baseline is introduced in this project between a **Human Planner** (static/reactive) and an **AI Orchestrator** (dynamic/proactive).

- **Cost of Inaction (CoI):** The financial penalty of not changing routes during a shock is defined to include:
  - War-risk insurance premiums.
  - Congestion surcharges.
  - Overtime warehouse labour.
- **Friction Propagation & Decay:** Disruptions do not remain isolated; they cascade. This is modelled via cascade multipliers:

$$\text{Severity}_{\text{cascaded}} = \text{Severity}_{\text{direct}} \cdot \text{Decay Factor} \tag{2.3}$$

For instance, a blockade in the Red Sea cascades to port congestions in Western

Indian ports (Mundra, Nhava Sheva) with a decay factor (e.g., 0.60).

2.4 Multi-Agent Systems (MAS) & LLM Orchestration

Instead of using traditional rigid coding paradigms to solve exceptions, the project leverages **Agentic AI** using the **CrewAI** framework:

- **Role-Playing Agents:** Labour is divided into isolated, specialised cognitive agents:
  - **(i)** *Logistics Optimiser (The Solver):* Mathematical routing is performed.
  - **(ii)** *Cost & Strategy Analyst (The Evaluator):* Tactical inventory checks and replenishment are managed.
  - **(iii)** *SME Communicator (The Presenter):* Complex telemetry is translated into visual dashboard payloads.
- **Prompt Chaining & Sequential Context:** The outputs of mathematical tools (Dijkstra) are piped sequentially as context to downstream agents. This prevents hallucination by feeding structured JSON outputs directly into the reasoning loop of the next agent.

2.5 Strategic Logistics Digital Twin

A Digital Twin is a dynamic digital representation of a physical system. In a strategic context, it is used for:

- **Concept of Strategic Digital Twin:** Unlike operational digital twins that focus on short-term execution, a Strategic Digital Twin is designed for long-term scenario planning and risk assessment. A "sandboxed" version of the supply chain network is created to simulate "What-If" scenarios—such as geopolitical blockades or sudden maritime congestion.
- **Application:** By injecting synthetic events (such as the simulated benchmark shocks), the Digital Twin acts as a risk simulation sandbox, allowing managers to visualise the downstream effects on landed costs and stockout timings before they occur in the real world. This transforms supply chain management from a reactive effort into a proactive resilience strategy.

2.6 Summary

This chapter provided a comprehensive overview of the five theoretical pillars that form the backbone of the project. By integrating stochastic inventory models with dynamic network flow and agentic orchestration, the framework is designed to address the "Cost of Inaction" in volatile maritime corridors. These concepts set the stage for the detailed system design and implementation phases discussed in the following chapters.

Chapter 3

Software Requirement Specification

CHAPTER 3

SOFTWARE REQUIREMENT SPECIFICATION

Detailed requirements and specifications of the proposed system are described in this section. The functionalities the system must perform, the performance expectations, operational constraints, and the hardware and software resources required for development and execution are defined herein. The Software Requirement Specification (SRS) serves as a formal reference document for system design, development, testing, deployment, and maintenance.

3.1 Overall Description

The proposed Agentic AI framework is designed as a proactive decision-support and execution system for Indian Small and Medium Enterprises (SMEs) engaged in global trade. The overall system provides a bridge between high-level geopolitical disruptions and ground-level logistics operations. By integrating unstructured data signals with a mathematical digital twin, the system enables SMEs to visualize risk and execute mitigation strategies—such as rerouting shipments or adjusting safety stocks—that were previously only accessible to large-scale enterprises with massive logistics departments.

The system is structured as a modular pipeline that begins with the **Friction Sentry**, which continuously parses and quantifies unstructured signals from news and maritime alerts. These quantified shocks are then injected into the **Networked Digital Twin**, which uses graph-theoretic algorithms like Dijkstra's to calculate the propagation of friction across the supply chain. The **Logistics Optimizer**, powered by a multi-agent orchestration layer (CrewAI), evaluates these impacts to generate actionable recovery plans. Finally, the **Interactive Dashboard** provides a geospatial interface for users to review these plans and perform "What-If" simulations. This comprehensive approach ensures that Indian SMEs can maintain operational continuity and cost-competitiveness even in the face of significant maritime chokepoint disruptions.

3.1.1 Product Perspective

The proposed system is an agentic, event-driven Logistics Digital Twin and Decision Support System (DSS) designed to simulate real-world supply chain shocks, such as military blockades, customs backlogs, and weather delays. It utilizes cooperative Large Language Model (LLM) agents to dynamically calculate optimal alternative shipping

routes and manage safety stock. Built as a modular, integrated client-server application, the system consists of a React-based frontend dashboard, a FastAPI backend service, an optimization engine powered by NetworkX, and an agentic orchestration layer using CrewAI and Google Gemini.

In terms of external interactions, the system utilizes the Google Gemini API as its primary LLM endpoint for parsing shocks, evaluating inventory states, and generating intelligent recommendations. For geospatial representation, it interacts with mapping services including OpenStreetMap and CartoDB via Leaflet to render spatial shipping routes. The operational environment is defined by a Python runtime (v3.10 or higher) on the backend and modern web browsers like Chrome, Edge, and Firefox for the React (Vite) single-page application frontend. Within the broader domain, the system fits into modern Supply Chain Risk Management (SCRM) by bridging the gap between traditional mathematical logistics routing and cognitive decision-making, allowing SMEs to proactively bypass global chokepoint delays.

3.1.2 Product Functions

The core functionalities of the system include:

- **Shock Signal Parsing & Translation:** Translates raw textual or unstructured disruption notifications (e.g., news alerts) into structured, quantitative shock parameters such as location, mode, and severity multipliers using semantic mapping rules.
- **Dynamic Dijkstra Route Optimization:** Re-evaluates the directed graph topology under a shocked state to calculate the mathematically shortest path based on specific cost or time preferences.
- **Real-time Inventory Monitoring:** Tracks the daily inventory status, consumption rates, and Days of Cover (DoC) for target SME markets within the networked digital twin.
- **Autonomous Stockout Prevention:** Detects impending stockouts during transit delays and automatically executes simulated emergency restocking procedures.
- **Human vs. AI Comparative Benchmarking:** Runs deterministic pipelines comparing a reactive human baseline (subject to the "Cost of Inaction") against the proactive agentic AI response, exporting comparative datasets for business in-

telligence.

- **Geospatial Route Visualization:** Renders baseline routes alongside AI-recommended alternate paths on an interactive geographical dashboard.

3.2 Specific Requirements

This section describes the detailed requirements of the proposed system, detailing both the functional capabilities needed to support autonomous logistics decision-making and the non-functional criteria required for system reliability. The specifications detailed here serve as a technical blueprint for the subsequent architecture and database design phases.

3.2.1 Functional Requirements

- **FR1: Knowledge Graph Construction**: The system must allow the creation of a directed graph mapping international ports to inland Indian SME hubs using NetworkX, supporting dynamic updates to edge weights.
- **FR2: Geopolitical Signal Ingestion (Friction Sentry)**: The system must utilize an LLM agent to ingest unstructured text and extract specific disruption parameters: Location, Severity Level, and Estimated Delay.
- **FR3: Friction Propagation**: The system must inject extracted delay parameters into the network graph to simulate a "shock" and calculate the cascading downstream impact on SME inventory.
- **FR4: Autonomous Optimization**: Upon detecting a critical friction threshold, the Logistics Optimizer agent must autonomously execute a shortest-path algorithm to output a specific, actionable alternative route.
- **FR5: Comparative Baseline Calculation**: The system must mathematically compute the "AI Landed Cost" and compare it against a pre-programmed "Human Baseline Cost" (using Chopra inventory formulas) to quantify savings.

3.2.2 Non-Functional Requirements

- **NFR1: Performance**: Graph traversal and alternate route calculations should execute with minimal latency to provide real-time decision support.
- **NFR2: Scalability**: The multi-agent pipeline must be decoupled from the graph engine, ensuring the system can scale to include more ports or swap out LLM

models.

- **NFR3: Usability**: The final dashboard must be highly intuitive, designed specifically for SME managers lacking enterprise data infrastructure.

3.2.3 Software Requirements

- **Operating System**: Windows 10/11, macOS, or Linux (Ubuntu 20.04+).
- **Programming Language**: Python 3.9 or higher
- **AI & LLM Services**: Google Gemini API (for core natural language reasoning and agent logic).
- **Agentic Frameworks**: CrewAI and LangChain (for orchestrating the multi-agent system and tool calling).
- **Mathematical & Graph Modeling**: NetworkX (for building the Multi-Tier Logistics Digital Twin), alongside Pandas and NumPy for data structuring.
- **Visualization**: Tableau (for the interactive "What-If" scenario dashboard).
- **Development Environment (IDE)**: Visual Studio Code (VS Code)
- **Frontend tools**: React (v19.x), Axios (v1.x), Leaflet & React-Leaflet (v5.x)
- **Version Control**: Git and GitHub

3.2.4 Hardware Requirements

- **Processor**: Intel Core i5 / AMD Ryzen 5 or equivalent (Minimum); Intel Core i7 / Apple M1/M2 or higher (Recommended for faster multi-tier graph computations).
- **RAM**: 8 GB (Minimum); 16 GB or higher (Recommended for handling large NetworkX logistics graphs and local multi-agent processing simultaneously).
- **Storage**: 256 GB SSD (Minimum) to store datasets, Python environments, and project files.
- **Network**: High-speed, stable internet connection (Crucial for continuous, low-latency API calls to Google Gemini and real-time data ingestion).

3.3 Summary

This chapter established the Software Requirement Specification for the Agentic AI framework, detailing both functional and non-functional requirements. It provided a comprehensive overview of the hardware and software stack necessary for building a

resilient logistics digital twin. These specifications serve as the baseline for the high-level and detailed design phases that follow.

Chapter 4

High-Level Design

CHAPTER 4

HIGH-LEVEL DESIGN

The overall design and architecture of the proposed system are presented here. A high-level representation of the system structure, major modules, component interactions, and data flow within the system is provided. The High-Level Design (HLD) phase transforms the requirements specified in Chapter 3 into a conceptual blueprint that guides the implementation process. System organisation, architecture strategies, module interactions, and workflow representation are focused upon without going into implementation-level or database-level details.

4.1 Design Considerations

This section explains the important factors considered while designing the proposed system. The purpose of this section is to justify the architectural, procedural, and design decisions taken during system planning. These factors ensure that the software remains robust, efficient, and user-friendly under variable operational conditions.

4.1.1 General Considerations

Here are the general design factors rewritten as prescriptive principles and requirements considered during system development:

System Scalability

- **Independent Scalability:** The frontend visualization layer and the backend optimization engine must be decoupled to allow independent scaling under variable user loads.
- **Component-Level Extensibility:** The system architecture must support the addition of new logical nodes, pathways, or specialized agents without requiring changes to the core routing algorithms.
- **Adjacency-Based Network Expansion:** The network model must use flexible data structures so that the supply chain topology can grow without rewriting the backend codebase.

Reliability

- **Deterministic Fallback Protocols:** The system must incorporate automatic fallback logic (such as mathematical overrides) to guarantee a valid routing decision

even if external cognitive APIs experience downtime or high latency.

- **State Isolation:** The database and tracking metrics must run with strict session isolation to prevent state leakage and ensure execution results are reproducible.

Maintainability

- **Separation of Concerns:** Core system configurations (such as physical network topologies and mapping rule libraries) must be isolated from the source code, utilizing external data files (like JSON) to simplify updates.
- **Modular Codebase:** Code structures must be organized into logical packages with distinct folders for routing engines, API endpoints, and benchmark scripts.

Security

- **Credential Isolation:** The storage of third-party API keys and server credentials must be restricted to local environment files rather than hardcoded in the codebase.
- **Access Control & Sandboxing:** Cognitive agents must operate with restricted permissions, modifying system variables or accessing database records only through structured, predefined interfaces.

User-Friendliness

- **Geospatial & Visual Simplification:** Complex textual routes and network logs must be represented as interactive, spatial visual pathways.
- **Consolidated Telemetry:** Key operational metrics (such as delays, financial deltas, and inventory warnings) must be displayed via clear, intuitive visual widgets.

Performance Efficiency

- **Low-Latency Calculations:** Graph-theoretic and mathematical routing calculations must operate in-memory to ensure response times remain in milliseconds.
- **Minimal Payload Overheads:** Context passed to external reasoning APIs must be token-optimized to minimize latency and call costs.

Modularity

- **Phase-Based Architecture:** The project pipeline must be divided into separate, independent modules for signal extraction, agentic orchestration, and mathematical execution so that each phase can be tested and modified in isolation.

Reusability

- **Interface Standardization:** Functions, utilities, and components must be generic and reusable across different simulation scenarios.

Flexibility

- **Dynamic Parameter Customization:** The system must allow users to toggle operational priorities (such as cost vs. time) and adjust simulation parameters dynamically during execution.

4.1.2 Development Methods

This subsection explains the software development methodology adopted for the project. The project follows a 6-week sequential execution timeline, divided into five core modules. Each module corresponds directly to a layer in the system architecture.

Given the deeply interconnected nature of the system components and the requirement for a stable mathematical baseline, the project utilizes a **Sequential Waterfall Handover Strategy**. This model was selected to ensure that each critical layer of the logistics digital twin—from network topology to agentic reasoning—is rigorously validated and frozen before downstream logic is applied. This prevents cascading errors in the mathematical optimization engine.

The development methodology is divided into the following phases:

- **Stage 1: Digital Twin Construction (Week 1)**: Mapping global maritime nodes to domestic SME hubs using `networkx`; establishing baseline edge weights such as transit time and logistics cost.
- **Stage 2: Domain Logic & Data Generation (Week 2)**: Quantifying "Grey Zone" friction variables (insurance spikes, port delays) and building the Semantic Mapping Library to link geopolitical keywords to specific graph nodes.
- **Stage 3: Agentic Intelligence & Monitoring (Week 3)**: Developing the *Friction Sentry Agent* using LangChain and CrewAI to autonomously ingest unstructured news data and extract critical location and severity metrics.
- **Stage 4: Autonomous Mitigation & Execution (Weeks 4–5)**: Developing the *Logistics Optimizer Agent*; injecting friction shocks into the graph and executing shortest-path algorithms (Dijkstra's) to calculate optimal rerouting and dynamic buffer stocks.

- **Stage 5: Visualization & Baseline Validation (Week 6)**: Calculating the "Human Baseline" using Chopra inventory formulas and deploying an interactive dashboard to visualize AI cost-savings versus manual planning.

To manage this sequential flow effectively within a decoupled architecture, the team adheres to an **Immutable Handover Protocol**, where each module must pass unit-testing before being consumed by the next:

- **Network to AI (M1 to M3 & M2)**: The Network Architect hands over the finalized logistics graph, which serves as the immutable environment for all agents.
- **Domain to Monitoring (M3 to M2)**: The Domain Specialist hands over the Semantic Dictionary; the AI Engineer cannot finalize the parsing agent until keyword mappings are locked.
- **Monitoring to Visualization (M2 to M4)**: The AI Engineer passes the final validated JSON payloads to the Dashboard Analyst for front-end integration.

4.2 Architecture Strategies

This section explains the architectural and technological strategies adopted during system design. Selecting the appropriate programming paradigms and technology stacks is essential to support high-performance operations research and cognitive reasoning. The logic behind choosing specific backend, frontend, and integration frameworks is elaborated below.

4.2.1 Programming Language

The system utilizes a split-stack architecture, leveraging Python for the backend logic and React (JavaScript/ES6+) for the frontend visualization dashboard.

Python (Backend & Agentic Engine)

Python was selected as the core engine due to its dominance in Artificial Intelligence and operations research. It provides a seamless bridge between cognitive AI frameworks (CrewAI, LangChain) and scientific computing libraries (NetworkX). Its high-level syntax and asynchronous capabilities via FastAPI allow for rapid prototyping of complex multi-agent workflows and real-time graph optimization.

React & JavaScript (Frontend Dashboard)

React is employed to build the interactive User Interface. For a logistics digital twin, real-time telemetry and dynamic geospatial rendering are critical. React's component-based architecture and Virtual DOM enable efficient updates of complex map visualizations (via React-Leaflet) and performance metrics without full page reloads, ensuring a responsive user experience during simulation shocks.

4.2.2 Future Plans and Scalability

This subsection outlines the strategic roadmap for evolving the system from a high-fidelity prototype to a production-grade autonomous logistics engine.

- **Transition to Real-Time Data Pipelines:** While the current framework demonstrates autonomous reasoning on simulated or static historical data, future iterations will integrate **Live Telemetry Ingestion**. By connecting the *Friction Sentry Agent* to real-time APIs (e.g., Lloyds List for maritime news, port congestion trackers, and satellite Automatic Identification System (AIS) data), the system will move from manual simulation to a "Live Sentry" mode, providing under-60-minute detection-to-mitigation cycles for Exim logistics.
- **Multi-Dimensional Friction Modeling:** The architecture is designed to scale from single-point shocks to **Compound Risk Scenarios**. Future enhancements will focus on the agent's ability to process and correlate multiple simultaneous friction signals—such as a geopolitical blockade in a chokepoint occurring concurrently with a domestic labor strike. This involves developing cross-impact matrices where agents reason about how separate shocks propagate through the network and amplify total logistics cost.
- **Predictive "Friction Forecasting":** Leveraging the data captured during the current implementation, future versions could incorporate **Predictive Analytics Engines**. By training machine learning models on historical friction patterns, the system could transition from being reactive (mitigating current shocks) to proactive (recommending route adjustments based on high-probability future disruptions).
- **Cloud-Native Enterprise Scaling:** To handle global-scale supply chains with thousands of nodes, the system can be migrated to a **Kubernetes-based Microservices Architecture**. This would allow the agentic swarm to scale horizon-

tally, assigning dedicated agent clusters to specific geographic regions or product categories, ensuring low-latency optimization for multi-national logistics networks.

4.3 System Architecture

This section explains the overall architecture and structural organization of the proposed system. A layered block layout is utilized to segregate the presentation, API routing, network optimization, and multi-agent reasoning layers. This division ensures that each component can be developed, tested, and scaled independently.

4.3.1 High Level System Architecture

The High Level System Architecture as shown in Figure 4.1 illustrates the multi-layered flow of information through the system, from raw data ingestion to final executive visualization.

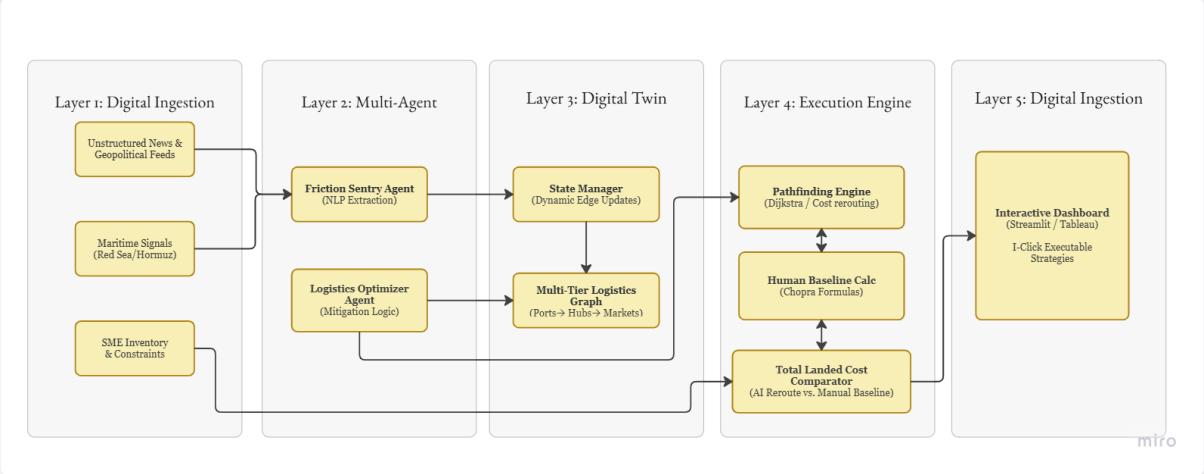

Figure 4.1: System architecture diagram of the proposed system

The framework is designed as a modular, five-layer system that separates data ingestion, state management, autonomous intelligence, communication APIs, and presentation. Figure **??** conceptually shows this layered representation, and the details of each layer are explained below:

Layer 1: Data Ingestion (Data Inputs): This layer represents the static and dynamic files that serve as the project's source of truth. It contains:

- `network_model.json`: The serialized representation of the 14-node, 19-edge EXIM network, frozen after Phase 1.

- `simulated_friction_events.csv`: A historical and hypothetical dataset containing 50 logistics disruption scenarios.
- `semantic_mapping_rules.json`: Predefined rules mapping unstructured signal types to quantitative severity multipliers and propagation decay parameters.

Layer 2: Digital Twin State (In-Memory Graphs): This layer maintains the virtual representation of the supply chain using two NetworkX directed graphs (`DiGraph`):

- *Baseline Graph:* An immutable graph representing normal operating conditions, loaded once at system startup.
- *Shocked Graph:* A dynamic deep copy of the baseline graph. Upon shock injection, the Friction Sentry agent mutates the edge weights of this graph to simulate real-time delays and cost spikes.

Layer 3: Autonomous Agent Layer (CrewAI Orchestration): This layer contains the four specialized CrewAI agents (Friction Sentry, Logistics Optimizer, Cost Analyst, and SME Communicator). Powered by Google Gemini, they execute sequential tasks to analyze shocks, run routing algorithms, check inventory levels, and generate management recommendations.

Layer 4: API and Orchestration Layer (FastAPI Backend): A high-performance FastAPI server serves as the system's integration hub. It exposes REST API endpoints that receive disruption signals from the frontend, trigger the multi-agent crew, read and update the digital twin state, and return the aggregated JSON payload back to the dashboard.

Layer 5: Presentation and Analytics Layer (React Dashboard): A responsive React/Vite frontend dashboard provides an interactive control panel for human managers. It includes Leaflet maps for visual path rendering, real-time alert logs, comparison panels for cost/time deltas, and indicators of agent decision confidence.

4.4 Data Flow Diagrams

Data Flow Diagrams (DFDs) represent the movement and processing of data within the system. They illustrate how external signals are parsed, how graph states are mod-

ified, and how optimized recommendations are transmitted to the dashboard. The following subsections detail the data flow across three distinct levels of abstraction.

4.4.1 Data Flow Diagram Level 0(Context Diagram)

The Context Diagram (Level 0) as shown in Figure 4.2 represents the highest-level view of the system, showing its boundaries and the primary external entities (Global Maritime Environment and SME Logistics Manager) that interact with the Agentic Logistic Network.

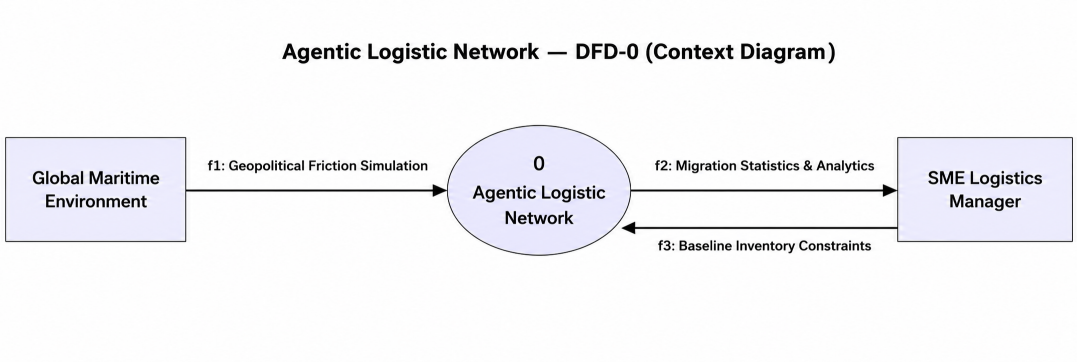

Figure 4.2: Data Flow Diagram Level 0 (Context Diagram)

4.4.2 Data Flow Diagram Level 1

The Level 1 DFD as shown in Figure 4.3 decomposes the system into its primary functional processes, illustrating the flow of data between knowledge graph construction, signal ingestion, shock propagation, and mitigation execution.

4.4.3 Data Flow Diagram Level 2

The Level 2 DFD provides a granular view of the Autonomous Mitigation process (Process 4.0) as shown in Figure 4.4, detailing the sub-processes involved in identifying risks, pathfinding optimization, and strategy assembly.

4.5 UML Diagrams

Unified Modeling Language (UML) diagrams are presented in this section to illustrate the behavioral and structural characteristics of the system. In particular, the sequence of operations and chronological message passing between frontend, backend, and external APIs are detailed. This representation helps visualize the execution lifecycle of a simulation shock.

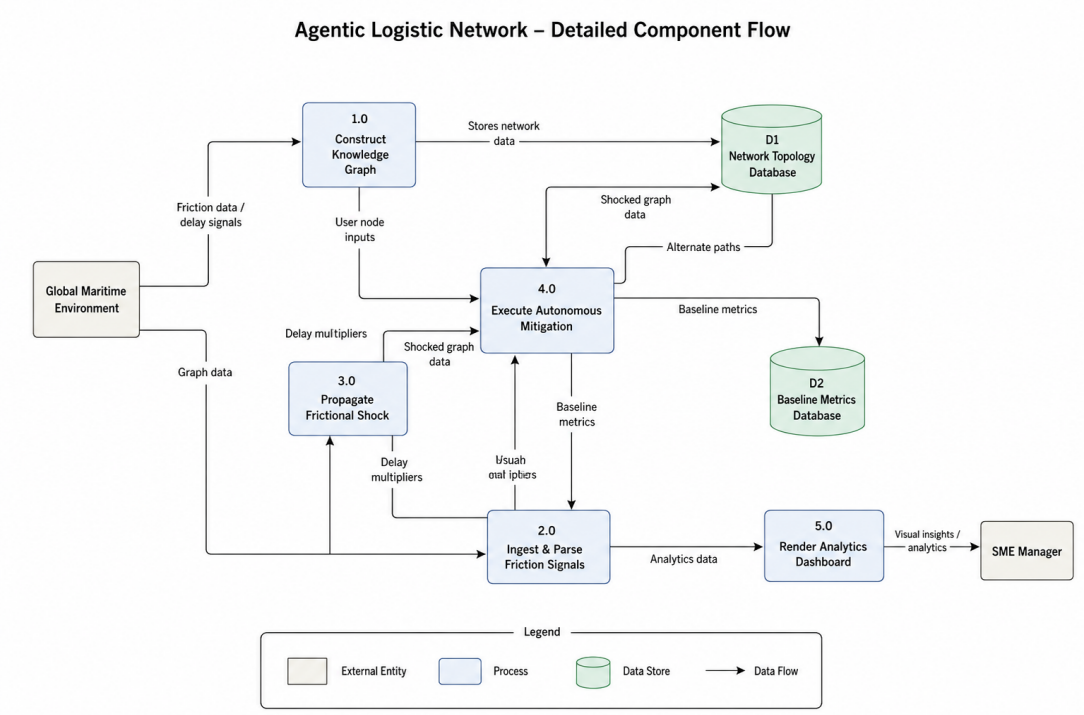

Figure 4.3: Data Flow Diagram Level 1

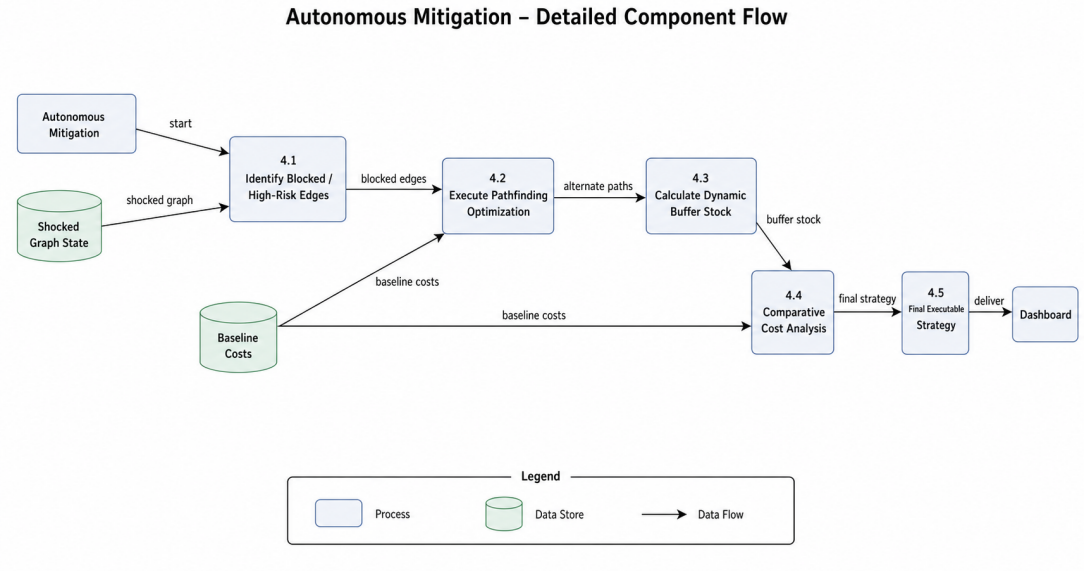

Figure 4.4: Data Flow Diagram Level 2

4.5.1 Sequence Diagram

The Sequence Diagram illustrated in Figure 4.5 shows the chronological flow of messages and operations across the system lifecycles, from initialization to final analytics delivery. The sequence begins when the **Client application (Frontend UI)** issues an execution request upon a simulated or detected chokepoint disruption. This event triggers an HTTP POST request containing location data and disruption details to the **FastAPI Web Server**. The Web Server validates the incoming payload and invokes the **Friction Sentry** module to translate the real-world textual event details into numerical severity metrics. The Friction Sentry maps the geographic location to a specific node within the logistics network topology and calculates a cascading friction multiplier based on chokepoint vulnerability tiers. Once this calculation is complete, a structured shock payload is returned to the FastAPI Web Server, which then forwards the state changes to the **Networked Digital Twin**. The Digital Twin applies these updates to its in-memory network representation, altering the travel time and shipping costs of the affected edges. Simultaneously, it evaluates the downstream impact on inventory levels, computing the Days of Cover (DoC) for the target destination markets. If the calculated DoC falls below safety thresholds, the Digital Twin alerts the Web Server, which triggers the **Logistics Optimizer** to execute routing optimization. The Optimizer queries the Digital Twin for the current state of the network graph and executes Dijkstra's pathfinding algorithm to calculate the baseline mathematically optimal path. It then forwards the computed path, inventory shortages, and node metrics to the **CrewAI Orchestrator** to initiate the agentic reasoning phase. The CrewAI Orchestrator invokes three specialized agents: the *Logistics Analyst Agent*, the *Logistics Optimizer Agent*, and the *SME Communicator Agent*. The Analyst Agent assesses geopolitical risks and potential cascading failures; the Optimizer Agent validates the path constraints and assesses the Cost of Inaction (CoI) to determine emergency replenishment triggers; and the Communicator Agent formats these insights into a structured, actionable logistics advisory. These agents query tools (such as routing and restocking databases) and call the external **Gemini LLM** to interpret risk profiles. Once the agents reach a consensus, the final advisory is serialized as a JSON object, returned through the Web Server, and rendered on the user's **Interactive Dashboard**.

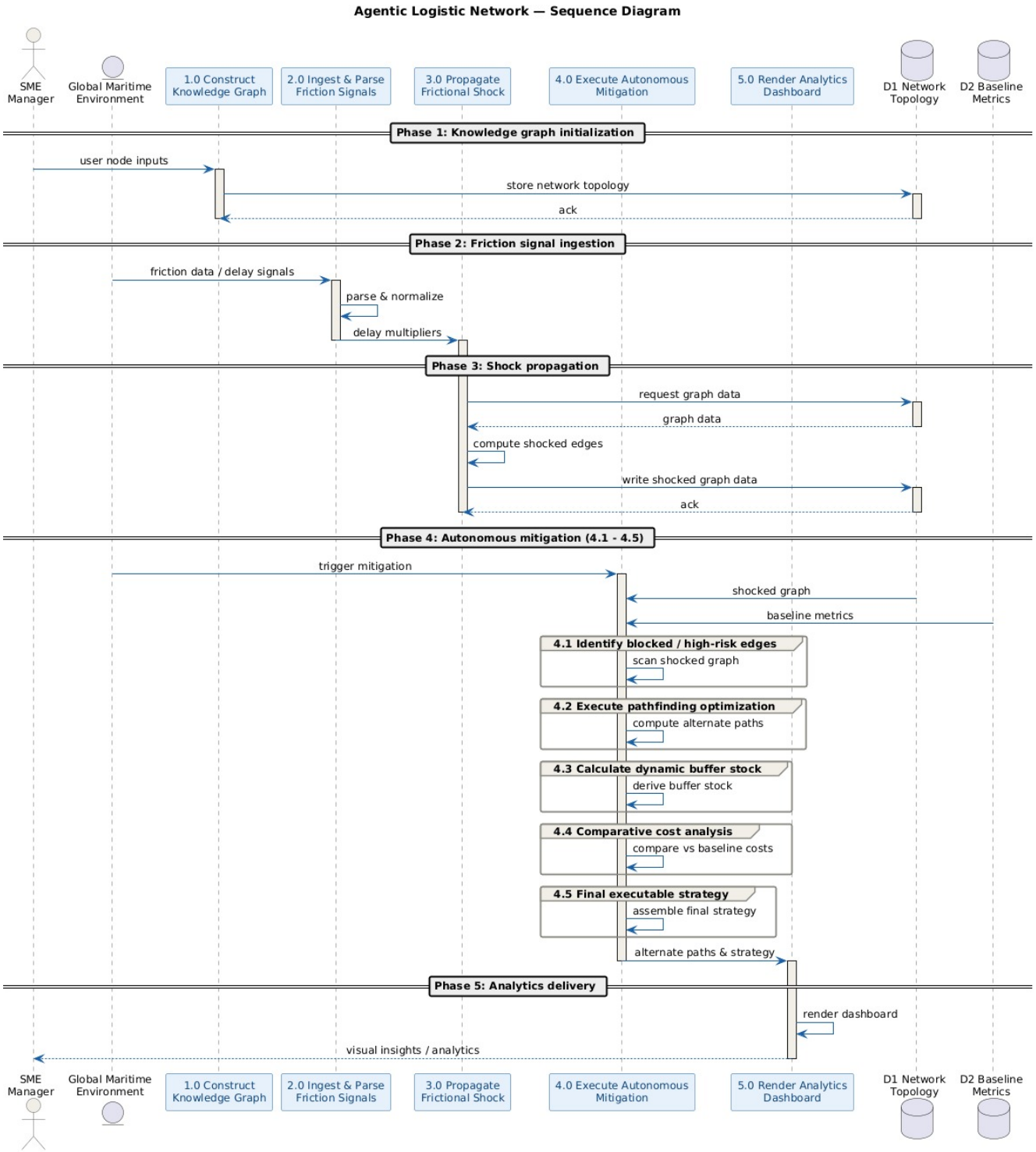

Figure 4.5: Sequence diagram illustrating the message and control flow between system components during shock simulation

4.6 Summary

This chapter detailed the High-Level Design of the system, illustrating the modular architecture and data flow between the intelligence, optimization, and visualization layers.

It established the core strategies for scalability and reliability, ensuring the framework can handle dynamic supply chain shocks effectively. The architectural blueprint provided here guides the detailed module development discussed in the next chapter.

Chapter 5

Detailed Design

CHAPTER 5

DETAILED DESIGN

The detailed design of the proposed system is presented here. The internal organisation of modules, the workflow of system operations, the database structure, interface design, and the detailed interaction between components are explained herein. Unlike the High-Level Design chapter, which focuses on the overall architecture and conceptual organisation of the system, this section focuses on the internal structural and functional design of the system components.

5.1 Structure Chart

This section presents the structural decomposition and organization of the system into modules and sub-modules. By breaking the digital twin application down into self-contained components, we ensure high cohesion and low coupling across the codebase. The functional divisions and control hierarchy are detailed in the following subsections.

5.1.1 Functional Decomposition

To manage complexity, the system is decomposed into four distinct, logical modules. Each module encapsulates a specific domain of operations, ensuring high cohesion and low coupling.

A. Friction Sentry Module

i. Purpose

This module is responsible for the ingestion, parsing, and structured interpretation of disruption signals. It serves as the risk-monitoring interface that maps raw event alerts to concrete physical impacts.

ii. Contribution to Overall System

It protects the downstream optimization algorithms from dealing with raw text or unstructured data by translating news signals into standardized variables (severity multipliers, location targets, and cost impacts).

B. Networked Digital Twin Module

i. Purpose

This module maintains the in-memory representation of the logistics network (directed graph topology) and models inventory health. It simulates the physical reaction of the

supply chain network when a shock is injected.

ii. Contribution to Overall System

It computes the real-world impact of delays on safety stock and inventory levels. By calculating stockout risks and daily consumption depletion, it provides the operational context needed to trigger emergency actions.

C. Logistics Optimizer Module

i. Purpose

This module serves as the execution and decision-making engine of the system, responsible for mathematical path routing and orchestrating autonomous LLM agents to resolve supply chain exceptions.

ii. Contribution to Overall System

It acts as the system's "brain." It calculates alternate paths bypassing disruptions, coordinates agent tasks (Optimizer, Analyst, and Communicator), and executes corrective replenishment actions to prevent stockouts.

D. Interactive Dashboard Module

i. Purpose

This module provides the user interface (UI) to visualize the state of the logistics network, display alternate route simulations, and present comparative benchmark metrics.

ii. Contribution to Overall System

It makes the complex underlying data understandable to human operators by overlaying geographic routes on a map, displaying stockout warnings, and highlighting the cost and time savings of agentic optimization versus the human baseline.

5.2 Module Interaction Flow

Modules interact asynchronously and sequentially to process disruptions and update the digital twin. Figure 5.1 depicts the sequential flow of data and control signals across the system components.

- **Triggering of Operations:** System execution is event-triggered, initiated either by a simulated news alert or by the user selecting a strategic preference (cost/time) and clicking "Run Optimization" on the dashboard.
- **Data Exchange Flow:**
    1. The API requests rules parsing.

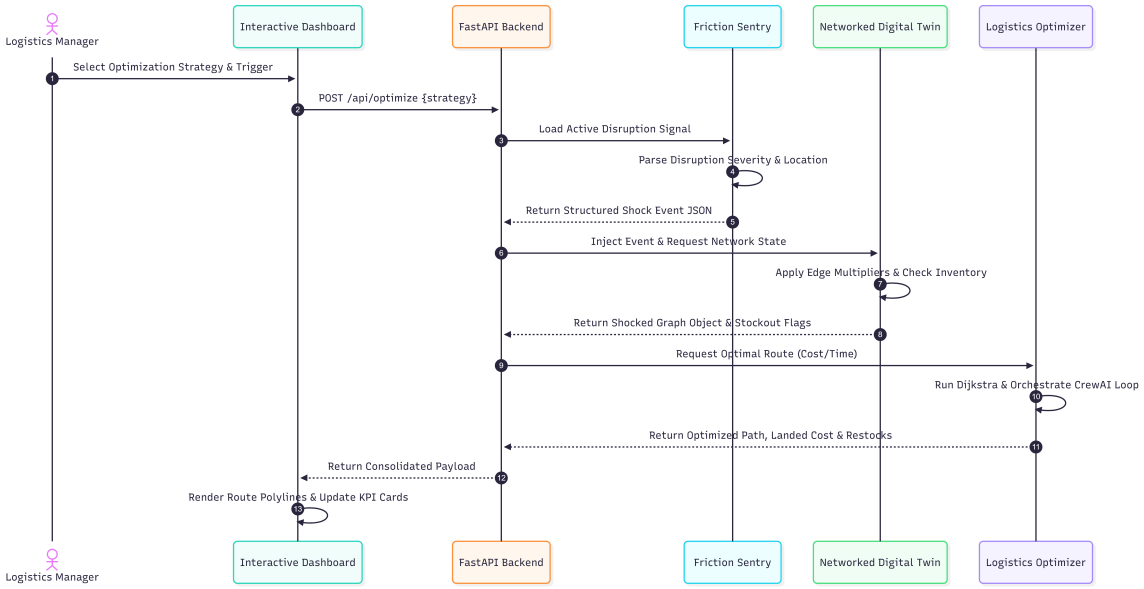

Figure 5.1: Detailed Module Interaction and Data Flow

2. The parsed rules feed directly into the graph-modification engine to create a shocked state.
3. The shocked state is evaluated for stockout risks.
4. The shocked parameters and inventory metrics are passed as prompt context to the `CrewAI` reasoning agentic loop.
5. The final dashboard payload is returned in a unified JSON structure.

5.3 Module Description

Detailed functional descriptions of each core module are provided in this section. For each module, the operational objective, inputs, internal processing steps, outputs, and dependencies are specified. This structured breakdown details the exact role that each module plays in the end-to-end simulation pipeline.

5.3.1 Module 1: Friction Sentry

- **Module Name:** Friction Sentry
- **Objective/Purpose:** To ingest unstructured disruption logs and map them to standard risk variables and propagation parameters.
- **Inputs:**
  - Raw event signal properties (location, event type, severity).

  - Static translation database (`semantic_mapping_rules.json`).

- **Internal Processing Performed:**
  - Maps free-text locations to valid graph node IDs.
  - Determines severity tiers (low, medium, high, critical) and extracts corresponding cost multipliers.
  - Calculates cascaded disruptions to adjacent nodes using decay rates (e.g., cascading Strait of Hormuz shocks to Nhava Sheva port with a 0.50 decay multiplier).
- **Outputs:** Structured JSON shock event payload.
- **Dependencies:** `json`, `re`.
- **Interaction with Other Modules:** Feeds structured shock data directly into the Networked Digital Twin to apply edge changes.

The logical flow of signal processing and validation within the Friction Sentry is illustrated in Figure 5.2.

Flowchart: Friction Sentry Module

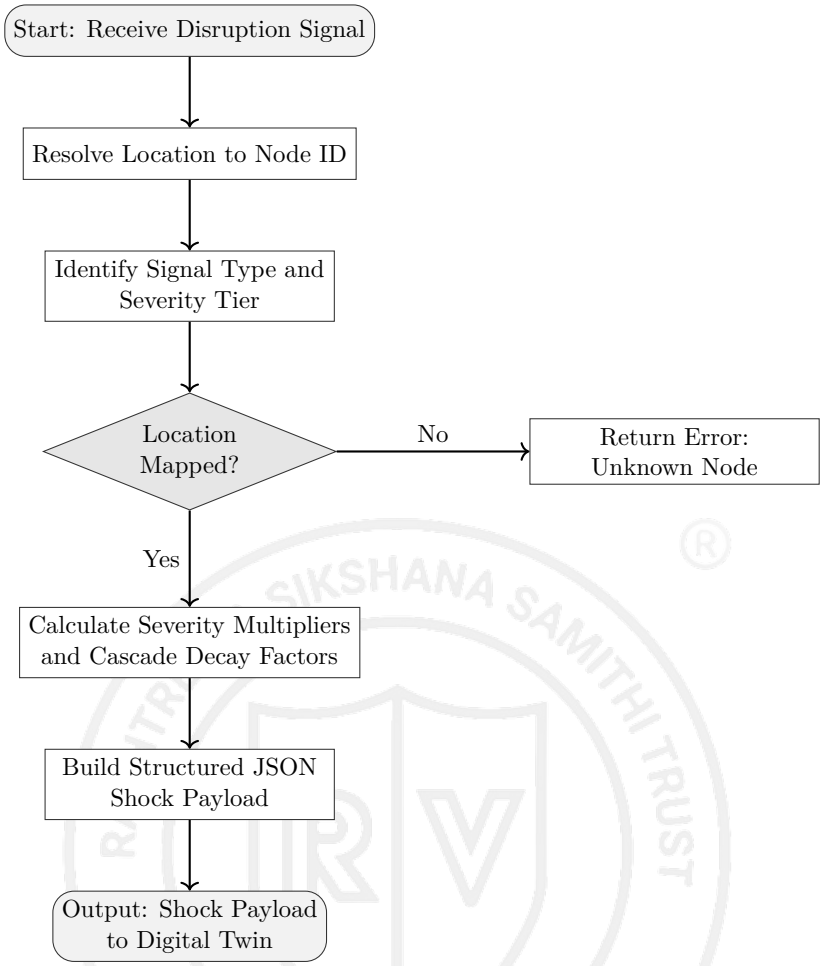

Figure 5.2: Flowchart of Friction Sentry Module

5.3.2 Module 2: Networked Digital Twin

- **Module Name:** Networked Digital Twin
- **Objective/Purpose:** To simulate the impact of network shocks on transit lanes and track target market safety stock parameters.
- **Inputs:**
  - Network topology configuration (`network_topology.json`).
  - Structured shock event payload (from Friction Sentry).
- **Internal Processing Performed:**
  - Modifies directed graph edge weights for cost and transit times according to severity multipliers.
  - Evaluates inventory Days of Cover (DoC) for target markets.

  - Executes the Chopra safety stock mathematical formulas to assess stock holding costs under lead time uncertainty.
  - Checks if transit delay exceeds inventory cover to raise stockout flags.

- **Outputs:** Shocked Network Graph Object (in-memory) and Inventory State Snapshot (utilization percentage, stockout Boolean).
- **Dependencies:** `networkx`, `InventoryManager` subclass.
- **Interaction with Other Modules:** Receives parameters from Friction Sentry and provides the active network state to the Logistics Optimizer for path-finding.

Flowchart: Networked Digital Twin Module

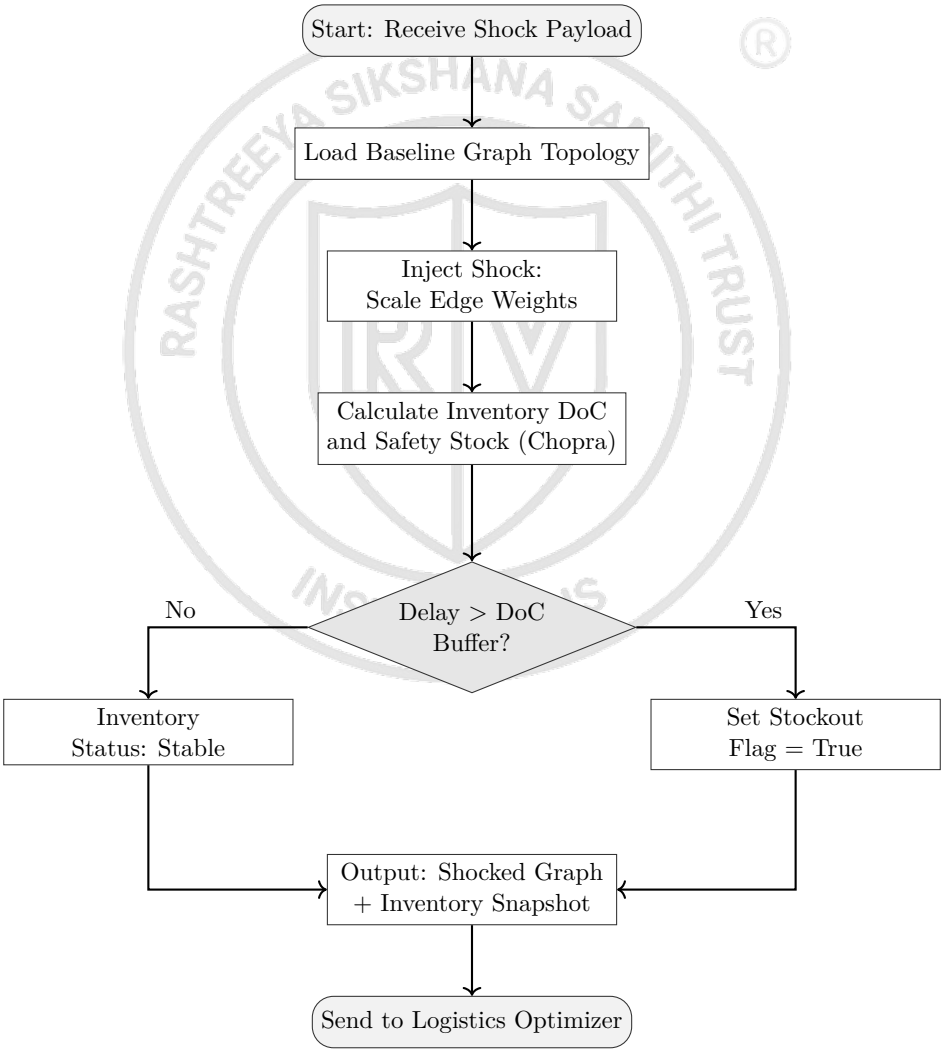

Figure 5.3: Flowchart of Networked Digital Twin Module

5.3.3 Module 3: Logistics Optimizer

- **Module Name:** Logistics Optimizer

- **Objective/Purpose:** To resolve network bottlenecks by computing optimal alternate paths and executing emergency inventory restocking actions.
- **Inputs:**
  - Shocked Network Graph Object and Target Destination ID.
  - Optimization Strategy selection (`cost` or `time`).
  - Remote LLM Connection (via Gemini API).
- **Internal Processing Performed:**
  - Executes Dijkstra's algorithm to find alternate paths that bypass blocked chokepoints.
  - Orchestrates a `CrewAI` execution pipeline (Logistics Solver → Cost Analyst → UX Communicator).
  - Triggers the `EmergencyRestockTool` to inject replenishment inventory if analyst agents identify an imminent stockout.
  - Benchmarks the dynamic optimized route against a static human baseline (subject to the "Cost of Inaction").
- **Outputs:** Optimized alternate route sequence, recalculated landed costs, optimized transit days, restock logs, and benchmark metrics.
- **Dependencies:** `crewai`, `networkx`, `google-genini-sdk`.
- **Interaction with Other Modules:** Receives graph structures from the Networked Digital Twin and sends calculation results to the Interactive Dashboard via API.

Flowchart: Logistics Optimizer Module

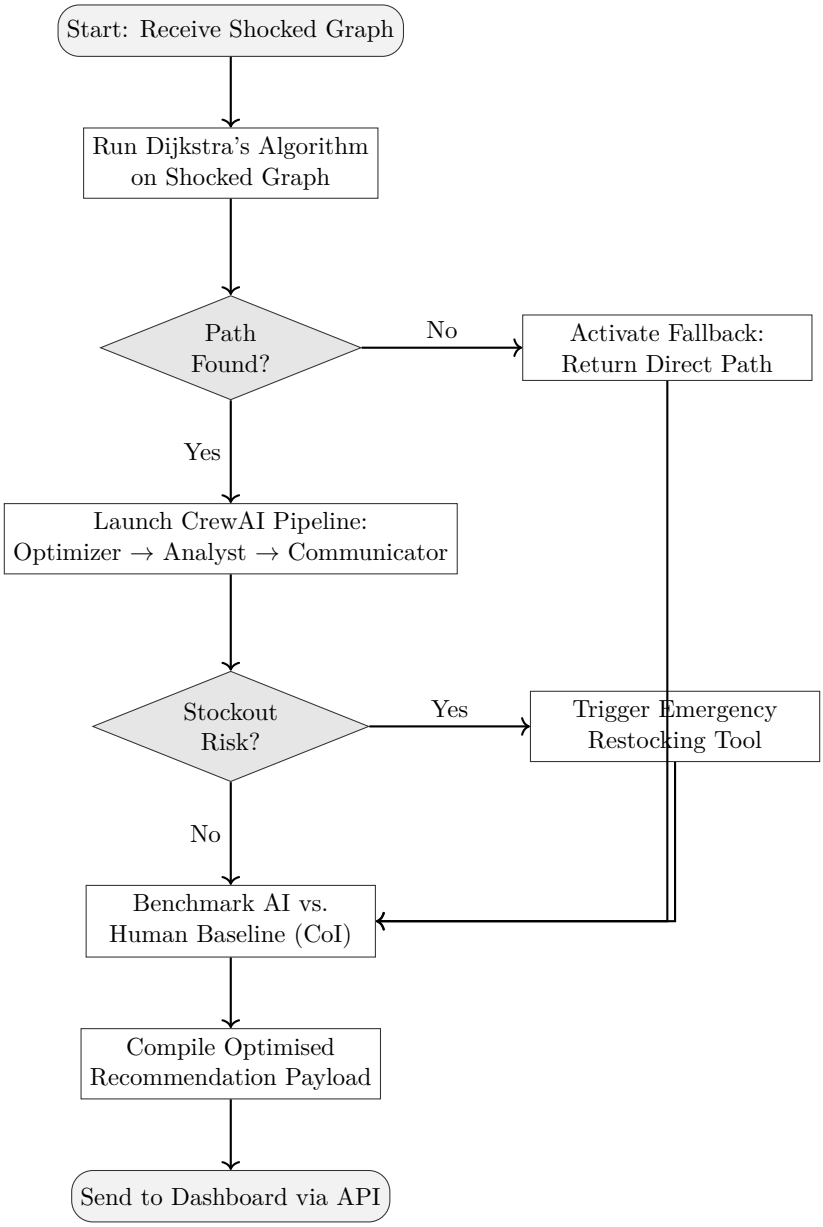

Figure 5.4: Flowchart of Logistics Optimizer Module

5.3.4 Module 4: Interactive Dashboard

- **Module Name:** Interactive Dashboard
- **Objective/Purpose:** To provide a visual user interface mapping spatial route modifications, telemetry data, and comparative benchmark results.
- **Inputs:**
  - Geographic coordinates dictionary (`COORDS`).
  - Optimization payload containing routes, metrics, and stockout statuses.

- **Internal Processing Performed:**
  - Translates node ID chains into latitude/longitude sequences.
  - Draws routing polylines (dashed for baseline routes, solid color-coded lines for optimized reroutes).
  - Renders alert banners, metric widgets (landed cost comparison, transit time changes), and stockout logs.
- **Outputs:** Interactive geographical web dashboard interface.
- **Dependencies:** `React`, `react-leaflet`, `axios`, `lucide-react`, `framer-motion`.
- **Interaction with Other Modules:** Triggers operations by sending optimization requests to the FastAPI backend and displays processing outcomes returned by the Logistics Optimizer.

Flowchart: Interactive Dashboard Module

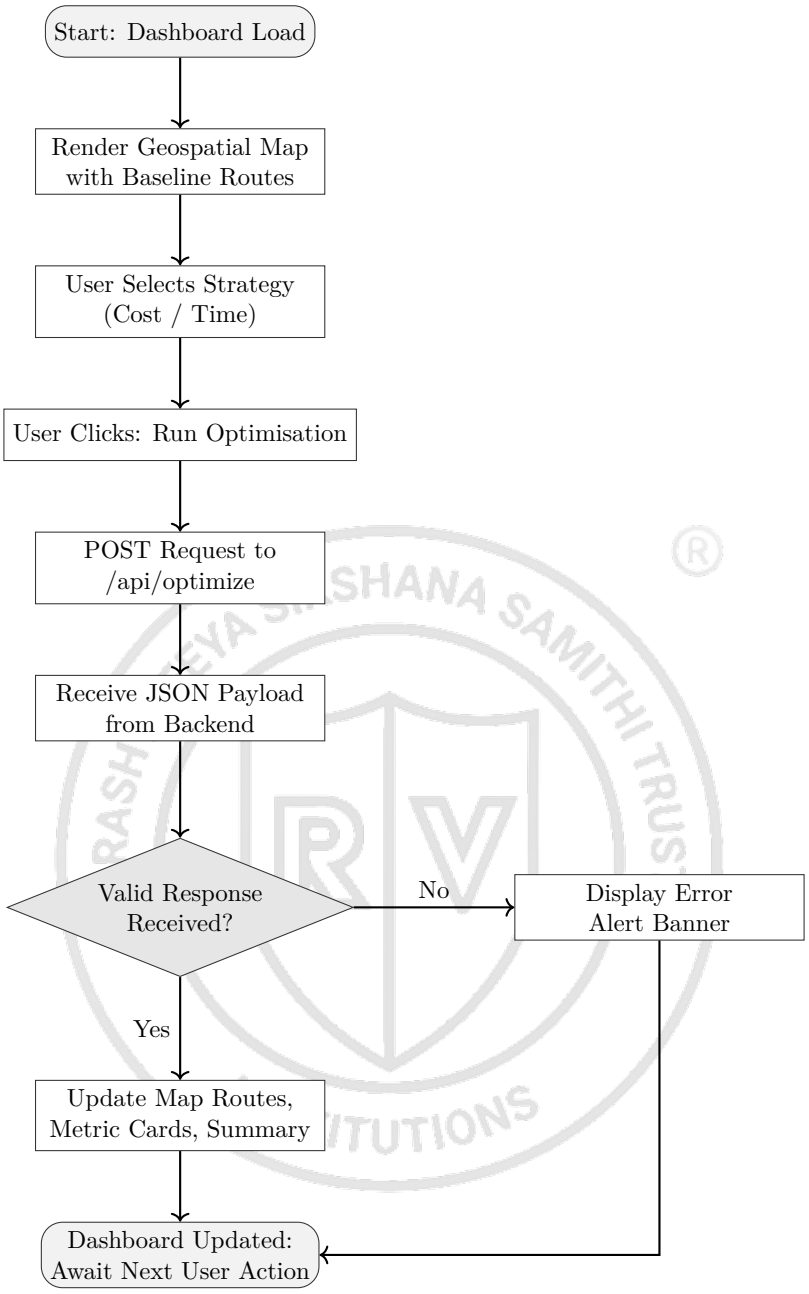

Figure 5.5: Flowchart of Interactive Dashboard Module

5.4 Database and Interface Design

The database and interface designs of the proposed system are detailed in this section. Managing a digital twin requires both structured document storage for network graphs and transactional flat-file recording for benchmark telemetry. The data organization strategies and user interface designs are described below.

5.4.1 Database Design

The system implements a hybrid data storage strategy tailored for digital twins and graph-theoretic applications.

- **Database Type:** The project utilizes a NoSQL Document Store (JSON configuration files) for managing the network graph structure and configuration rules, combined with a Flat-File Relational Database (CSV format) for logging analytical, tabular benchmark statistics.
- **Data Organization Strategy:**
  - **Network Topology Store (`network_topology.json`):** Formatted as a document store modeling a directed graph, organizing data into nodes (entities) and edges (relational links).
  - **Semantic Rules Library (`semantic_mapping_rules.json`):** Stores risk factors and propagation properties as structured, hierarchical key-value documents.
  - **Comparative Analysis Log (`comparative_analysis.csv`):** Stores structured, flat-file relational records comparing execution metrics of human vs. AI agents for dashboard ingestion.
- **Relationships between Entities:**
  - **Nodes and Edges:** A node represents a physical facility (Supplier, Port, Warehouse, or Market). Edges represent directional shipping lanes connecting a source node to a target node, containing attributes like baseline cost, baseline transit time, and mode.
  - **Events and Nodes:** A disruption event maps directly to one or more nodes (representing the location of the shock) and propagates along outgoing edges.
- **Storage Structure:** In-memory representation uses Python dictionary mappings and dynamic NetworkX graph structures, backed by persistent local file storage to minimize database connection overheads and maximize execution speed.

5.4.2 ER Diagram

The following diagram illustrates the logical relationships between the core operational entities modeled in the logistics digital twin:

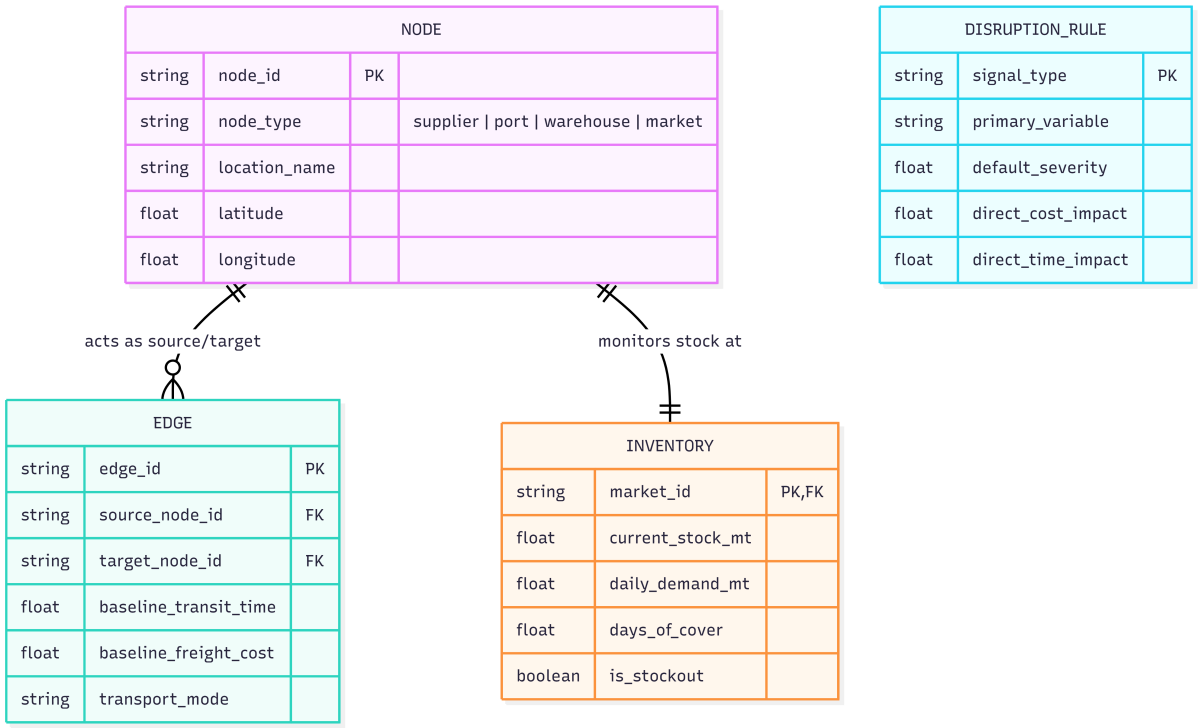

Figure 5.6: Entity-Relationship Diagram for Logistics Digital Twin

5.4.3 Table Structure Description

Entity: NODE

Purpose: Defines physical facilities and processing nodes within the logistics network, as shown in Table 5.1

Table 5.1: Node Entity Structure

Field Name	Data Type	Constraints	Field Purpose
`id`	String	Primary Key, Unique, Not Null	Unique identifier for the network node (e.g., `nhava_sheva`).
`type`	String	Enums: `supplier`, `port`, `warehouse`, `market`	Categorizes the operational nature of the facility.
`name`	String	Not Null	User-friendly geographical name (e.g., "Nhava Sheva Port").
`latitude`	Float	Range: -90.0 to 90.0	Latitude coordinate for geographical mapping.
`longitude`	Float	Range: -180.0 to 180.0	Longitude coordinate for geographical mapping.

Entity: EDGE

Purpose: Represents shipping lanes connecting physical nodes in the graph, as shown in Table 5.2.

Table 5.2: Edge Entity Structure

Field Name	Data Type	Constraints	Field Purpose
`source`	String	Foreign Key (references `NODE.id`)	Origin node of the logistics lane.
`target`	String	Foreign Key (references `NODE.id`)	Destination node of the logistics lane.
`transit_time`	Float	Value > 0	Baseline duration in days to traverse the lane.
`freight_cost`	Float	Value > 0	Baseline transportation cost in USD.
`mode`	String	Enums: `ocean`, `road`, `rail`	Transport modality utilized on the lane.

Entity: INVENTORY

Purpose: Tracks real-time stock parameters for destination nodes, as shown in Table 5.3.

Table 5.3: Inventory Entity Structure

Field Name	Data Type	Constraints	Field Purpose
`market_id`	String	Primary Key, Foreign Key (references `NODE.id`)	Identifies which market the inventory parameters apply to.
`current_stock_mt`	Float	Value ≥ 0	Quantity of stock currently on hand in Metric Tons.
`daily_demand_mt`	Float	Value > 0	Average daily inventory consumption rate.
`days_of_cover`	Float	Value ≥ 0	Estimated days remaining before stock is fully depleted.
`is_stockout`	Boolean	True / False	Flag showing whether on-hand inventory is completely depleted.

5.4.4 Interface Design

The user interface is designed as an interactive, single-page command-and-control dashboard structured to visualize real-time simulation data.

- **Screen Organization:**

  The layout is divided into three primary visual zones:

  a) **Top Banner (Notification Panel):** Renders critical shock alerts (e.g., location, event type, severity multiplier) detected by the Friction Sentry.

  b) **Left Panel (Geospatial Digital Twin Map):** Displays two visual maps in a split container: the *Baseline Network Map* showing undisrupted transit routes, and the *Optimized Alternate Route Map* highlighting the new routing paths calculated by the system.

  c) **Right Panel (Control & Telemetry Panel):** Hosts the strategy configuration controls, optimization triggers, and dynamic performance cards (e.g., Transit Time Delta, Cost Delta, Stockout Alarms, Executive Summaries).

- **Navigation Flow:**

  As a Single Page Application (SPA), the user does not leave the dashboard. The navigation flow is structural:

  Dashboard Load → Dynamic Map Renders → Configure Strategy (Cost/Time) → Click 'Run Optimization' → Executive Summary Slides In → Path Update Animates on Map

- **Input Forms / Control Elements:**

  – **Strategy Toggle:** Radio buttons allow the user to select the optimization objective (`Minimize Cost` vs. `Minimize Time`).

  - **Action Button:** A single high-contrast button labeled `Run Optimization` triggers the background agentic pipeline.

- **Output Displays:**
  - **Alert Banner:** Dynamic color-coded notifications showing active network shocks (e.g., "Verification Delay: Multiplier 1.35x").
  - **Dynamic Map Overlay:** Renders physical nodes (markers) and interactive polylines (dashed gray for baseline routes, bold purple for optimized agent routes).
  - **Metric Cards:** Renders computed values for the optimized paths, including *New Transit Days* and *Cost Delta*.
  - **Executive Summary:** A dynamic text block generated by the Communicator Agent explaining the reasoning behind the recommended actions.

5.4.5 API / Module Interface Design

The system facilitates module communication via a RESTful API layer managed by FastAPI using standardized JSON payloads.

API Interaction Workflow

- Communication between the React client and FastAPI server is handled over HTTP/HTTPS using standard CRUD operations.
- Dynamic optimization queries use `POST` requests containing configuration parameters in the request body.

Request-Response Structure

A. Shock Detection Endpoint (`GET /api/shock`)

- **Purpose:** Retrieves the active network shock parsed by the Friction Sentry.
- **Response Format (JSON):**

B. Optimization Trigger Endpoint (`POST /api/optimize`)

- **Purpose:** Runs the multi-agent optimization sequence based on user preference.
- **Request Body (JSON):**

```
{
  "optimize_by": "cost"
}
```

- **Response Format (JSON):**

```
{
  "recommended_path": "oman -> nhava_sheva ->
     mumbai_warehouse -> bengaluru_sme_market",
  "total_transit_days": 10.34,
  "total_cost_usd": 1695.96,
  "reroute_triggered": true,
  "summary_text": "AI selected the Oman-to-Bengaluru path via
      Nhava Sheva to avoid Red Sea shocks. Landed cost
     optimized to $1,695.96 with a 10.34-day transit delay.",
  "delta": {
    "time_change": "10.34 days",
    "cost_increase": "+$0.00"
  }
}
```

Communication Interfaces

- **CORS Middleware:** Backend APIs are configured with Cross-Origin Resource Sharing (CORS) policies to allow communication from the local React frontend port (e.g., `http://localhost:5173`).
- **Data Exchange Format:** All data exchanges are formatted as strict **application/json** payloads. Client-side state updates are triggered asynchronously upon resolution of the HTTP request promise.

5.5 Detailed Workflow and Processing

This section presents the detailed workflow and processing routines executed by the system during a simulation run. The lifecycle of a request involves input acquisition, validation, graph transformation, agentic reasoning, and output serialization. These sequential stages are explained in detail in the following subsections.

5.5.1 Input Processing Workflow

This subsection details how incoming requests and disruption signals are parsed and validated:

- **Input Acquisition:**

  The system acquires inputs from two channels:

  1. **User Action:** The frontend client transmits optimization preferences (e.g., `{"optimize_by": "cost"}`) via asynchronous HTTP POST requests.
  2. **External File Stream:** The Friction Sentry ingests shock events (event date, target node location, signal type, severity classification) from configuration inputs.

- **Input Validation:**

  Backend validation is handled programmatically at the API layer:

  - **Type Enforcement:** Inputs are validated using Pydantic models (e.g., checking that `optimize_by` matches one of the string constants `cost` or `time`).
  - **Bounds Verification:** Severity multipliers are checked to ensure they are numeric values ≥ 1.0.

- **Input Formatting:**

  Incoming raw strings are stripped of whitespace and converted to standardized, lowercase system identifiers. For example, user inputs or news locations (e.g., "Nhava Sheva") are mapped to graph node IDs (e.g., `nhava_sheva`).

- **Data Verification:**

  Before executing calculations, the backend verifies that the requested shock location and target market IDs exist in the active `network_topology.json` model to prevent reference key errors.

5.5.2 Internal Processing Workflow

This subsection details the processing stages executed once inputs are verified, as shown by Figure 5.7

- **Core Processing Stages:**

  The system executes a sequence of logical operations starting from input validation, injecting shock multipliers into the graph model, computing optimized paths via

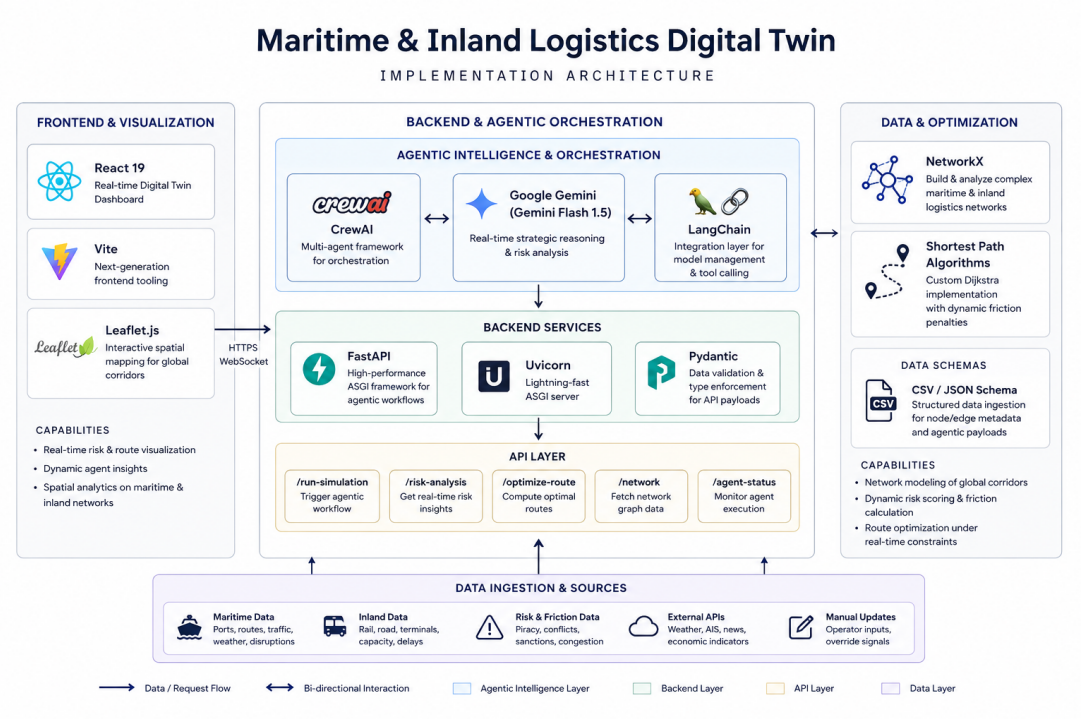

Figure 5.7: Internal Processing Workflow

Dijkstra's algorithm, orchestrating the CrewAI reasoning loop, and performing final inventory health checks.

- **Internal Data Movement:**
  - **Graph Model Loading:** The engine reads the baseline graph model and instantiates a `networkx.DiGraph` object.
  - **Shock Injection:** Edge weights (w_t and w_c) on the graph are scaled at the target locations using the shock severity multiplier.
  - **Path Optimization:** Dijkstra's algorithm computes the shortest path arrays (sequence of node IDs) based on the current active weight.
  - **Agent Prompt Construction:** The computed path sequences, delta days, and inventory snapshots are injected into the LLM system prompt templates for the CrewAI task configurations.
- **Module Coordination & Sequence:**

  Execution follows a strict sequential pipeline:

  Friction Ingestion → Graph Shock Calculation → Dijkstra Routing → Agentic Analysis & Stock (5.1)

5.5.3 Output Generation Workflow

This subsection details how the backend formats and delivers results:

- **Output Generation Process:**

  Upon completion of the agent tasks, the system coordinates the outputs of the *Optimizer*, *Analyst*, and *Communicator* agents.

- **Data Formatting:**

  Raw agent logs are parsed using regular expressions to isolate the outermost JSON block. Float values (such as costs and days) are rounded to two decimal places. Path sequences are formatted as clean string links (e.g., `Node A -> Node B -> Node C`).

- **Notification or Response Generation:**

  FastAPI converts the internal dictionary objects into a JSON HTTP response payload. Concurrently, the system updates the terminal logging console. The frontend intercepts this payload to trigger animations and update map polylines.

5.5.4 Exception and Validation Flow

This subsection explains how the system handles operational errors and exception states:

- **Validation Mechanisms:**
  - **Pydantic Schemas:** Detects invalid parameters at the REST endpoint level, returning HTTP 422 Unprocessable Entity errors.
  - **Reference Checks:** Custom validators verify that nodes parsed by the Friction Sentry exist in the active topology graph.
- **Error Handling Flow:**
  - **LLM API Timeout:** If the Google Gemini connection experiences high network latency or API rate limits, the system raises a connection timeout exception.
  - **Graph Disconnection:** If a shock completely isolates a destination node (blocking all paths), Dijkstra's algorithm will fail to find a path, raising a `NetworkXNoPath` error.
- **Recovery Mechanisms:**

- **Agent Route Fallback:** If the `CrewAI` process fails to output valid route JSON due to parser exceptions, the system activates a fallback handler that returns the mathematically computed Dijkstra path coordinates directly to the frontend.
- **Graceful API Degradation:** Under API errors, the UI replaces detailed summar" status ": " active ", ies with predefined, simplified alert messages while maintaining graph rendering.

5.6 Summary

This chapter provided a Detailed Design of the logistics digital twin, focusing on internal module logic, database schemas, and interface structures. It translated high-level architectural goals into specific execution workflows and API contracts for the multi-agent system. These detailed specifications ensure a cohesive integration between the backend optimization engine and the interactive frontend dashboard.

Chapter 6

Implementation Details

CHAPTER 6

IMPLEMENTATION DETAILS

Detailed information regarding the implementation of the proposed system is provided here. How the system design discussed in previous chapters was transformed into a working solution using appropriate technologies, tools, frameworks, and development practices is explained herein.

6.1 Implementation Requirements

This section specifies the hardware and software requirements necessary for the development and execution of the proposed system. Establishing a baseline environment ensures that the backend optimization engine, local agentic execution, and frontend leaflet dashboard operate smoothly and without resource bottlenecks.

6.1.1 Hardware Requirements

- **Processor**: Intel Core i5 / AMD Ryzen 5 or equivalent (Minimum); Intel Core i7 / Apple M1/M2 or higher (Recommended for faster multi-tier graph computations).
- **RAM**: 8 GB (Minimum); 16 GB or higher (Recommended for handling large NetworkX logistics graphs and local multi-agent processing simultaneously).
- **Storage**: 256 GB SSD (Minimum) to store datasets, Python environments, and project files.
- **Network**: High-speed, stable internet connection (Crucial for continuous, low-latency API calls to Google Gemini and real-time data ingestion).

6.1.2 Software Requirements

- **Operating System**: Windows 10/11, macOS, or Linux (Ubuntu 20.04+).
- **Programming Language**: Python 3.9 or higher
- **AI & LLM Services**: Google Gemini API (for core natural language reasoning and agent logic).
- **Agentic Frameworks**: CrewAI and LangChain (for orchestrating the multi-agent system and tool calling).
- **Mathematical & Graph Modeling**: NetworkX (for building the Multi-Tier Logistics Digital Twin), alongside Pandas and NumPy for data structuring.

- **Visualization**: Tableau (for the interactive "What-If" scenario dashboard).
- **Development Environment (IDE)**: Visual Studio Code (VS Code)
- **Frontend tools**: React (v19.x), Axios (v1.x), Leaflet & React-Leaflet (v5.x)
- **Version Control**: Git and GitHub

6.1.3 Development Environment

The development setup is designed to support the local execution, integration testing, and debugging of the digital twin application.

- **Development Workflow:**

  The development flow uses a standard client-server iteration cycle. Backend algorithmic changes are written in Python and instantly exposed via FastAPI hot-reloading. The frontend React interface consumes these APIs, validating structural updates on local development ports (e.g., `http://localhost:5173`).

- **Project Structure:**

  The workspace is organized into isolated directories:

  - `/logistics_app_backend/`: Hosts backend application logic (`main.py`), local environment settings, and API routes.
  - `/logistics_app_frontend/`: Contains the React project source code, dependencies (`package.json`), and build assets.
  - `/network_model/`: Stores static graph topologies and network definition files.
  - `/engine/`: Contains the core mathematical calculation scripts (Dijkstra algorithm executions and shock calculators).

- **Environment Configuration:**

  Secret keys (such as the Gemini API credentials) and server settings are managed using a `.env` file loaded into the system process at runtime, preventing configuration details from being committed to the repository.

- **Build and Execution Environment:**

  - **Backend:** Executed via Python 3.10+ virtual environments (`venv`), with package installations managed by `pip`. The REST API runs locally using `uvicorn` as the ASGI web server.
  - **Frontend:** Built and served locally using Vite's development server, which

handles fast ESbuild transpiling.

- **Testing and Debugging Environment:**
  - Interactive script executions (`python main.py`) are tested using API clients like **Postman** or **cURL** to verify route payloads.
  - Web console logs (Chrome Developer Tools) are used to track Leaflet map rendering issues and evaluate React state modifications.

6.2 Implementation Tool Features

The core language, library, and framework features leveraged during implementation are presented in this section. Modern software engineering relies heavily on using built-in language properties and optimized open-source libraries to manage complex graph traversals and asynchronous agent execution. The key features of our stack are described below.

6.2.1 Programming Language Features

Python

- **Dynamic Typing with Optional Type Hinting:** Offers development speed while allowing static checking via typing annotations (e.g., `list[dict]`), reducing variable errors.
- **First-Class Functions:** Functions can be treated as arguments, enabling the system to pass mathematical tools (`CheckInventoryTool`, `DijkstraOptimalRouteTool`) directly to `CrewAI` tasks.
- **Cross-Platform Compatibility:** The standard libraries (`pathlib`, `json`, `math`) compile and execute identically on Windows, Linux, and macOS.
- **Automatic Memory Management:** Internal garbage collection simplifies memory operations when dynamically modifying and discarding large graph models in memory.

JavaScript (ES6+ / JSX)

- **Asynchronous Event Loop:** Supports non-blocking operations (`async/await`), ensuring that long-running agent requests do not freeze the browser UI.
- **Component Declarativity (JSX):** Allows nesting HTML structures directly inside JavaScript logic, creating self-contained UI modules that represent network

nodes and paths.

6.2.2 Framework and Library Features

FastAPI (Backend Web Framework)

- **Asynchronous Support:** Built on ASGI standards, allowing high-concurrency requests.
- **Automatic Documentation:** Instantly compiles OpenAPI/Swagger documentation pages (`/docs`) for API testing.
- **Pydantic Validation:** Ensures incoming POST bodies are type-safe before internal execution.

CrewAI (Agentic Orchestration Framework)

- **Role-Playing Architecture:** Allows setting custom goals, backstories, and LLM temperature constraints for individual agents.
- **Collaborative Chains:** Pipes task contexts sequentially, feeding output states (such as Dijkstra calculation results) directly into downstream tasks.

NetworkX (Graph Theory Library)

- **Complex Graph Structures:** Provides class definitions for Directed Multigraphs (`DiGraph`).
- **Optimized Math Solvers:** Implements a highly efficient Dijkstra pathfinder to instantly resolve alternative lanes during disruptions.

React (Frontend UI Framework)

- **State Hooks (`useState`, `useEffect`):** Automatically triggers map and dashboard updates when data is received from backend APIs.
- **Virtual DOM:** Minimizes layout shifts, allowing the UI to render dynamic animations smoothly.

6.2.3 Database Features

The system relies on a **NoSQL Document Store** consisting of structured JSON documents to represent operational states:

- **Data Storage Capabilities:** Stores hierarchical, unstructured, or semi-structured data (nodes array, edges array, mapping keywords) without requiring rigid schema alterations.

- **Query Handling Support:** Graph nodes are queried in $O(1)$ constant time using Python dictionary hashing lookup schemes.
- **Scalability and Performance Features:** In-memory graph loading allows the system to execute path queries and shock weight calculations within milliseconds, bypassing standard database connection pooling latencies.
- **Data Management Capabilities:** Simple file-based configuration makes it easy to back up, restore, and transfer the entire logistics network state by copying text files.

6.2.4 Version Control and Supporting Tools

- **Version Control System (Git):** Used to track incremental code changes, manage code branches, and merge frontend/backend components.
- **Collaboration Tools (GitHub):** Serves as the central repository host, allowing developers to review code revisions, document issues, and manage project milestones.
- **Testing and Debugging Tools:**
  - **Chrome DevTools:** Used to inspect network request-response payloads and debug React rendering performance.
  - **Uvicorn Console Logs:** Outputs color-coded real-time backend API requests and stack traces.
- **Build Automation Tools (`npm` / Vite):** Manages package compiling, optimizes JavaScript assets, and minimizes code size for frontend builds.

6.3 Platform Selection and environment

This section discusses the platform selection and operating environment chosen for the proposed system. Designing a real-time digital twin requires careful balancing of client-side responsiveness and backend computational throughput. The operational runtime constraints and hosting choices are detailed below.

6.3.1 Platform Selection

The system is developed as a Web-Based Client-Server Application. This platform configuration was selected over desktop or standalone alternatives based on the following engineering rationales:

- **Compatibility with Project Requirements:** A logistics digital twin requires real-time geospatial visualization and interactive simulation controls. A web-based platform allows integrating open-source mapping engines (Leaflet) directly into the UI, rendering complex path coordinate shifts instantly inside the browser.
- **Performance Considerations:** By splitting the application into a frontend client and a backend server, heavy computational tasks—such as executing Dijkstra's pathfinder on dynamic graphs and orchestrating CrewAI reasoning loops—are offloaded to the backend. The client machine's resources are dedicated exclusively to UI rendering, preventing layout freezes.
- **Scalability:** A web platform facilitates easy scaling. The backend can be containerized and hosted on cloud environments to scale horizontally, while multiple client devices (logistics managers, warehouse operators) can access the dashboard simultaneously via web browsers.
- **Ease of Development:** Web standards (React, JavaScript, CSS) provide robust libraries for UI animations and responsive grids. This allows for rapid iteration of layout elements (such as the glassmorphic design and control panels) without rebuilding platform-specific installers.
- **Resource Availability:** Web development enjoys extensive documentation and open-source packages (e.g., standard mapping wrappers, HTTP client managers, and charting libraries), reducing integration overheads.

6.3.2 Operating Environment

The runtime and execution specifications of the system are defined as follows:

- **Supported Operating Systems:**
  - **Development & Hosting Server:** Cross-platform compatibility. Supported on Windows (10/11), macOS, and Linux (Ubuntu 20.04+).
  - **Client Devices:** Any OS equipped with a modern web browser, including Windows, macOS, Linux, iOS, and Android.
- **Runtime Environment:**
  - **Backend Runtime:** Python v3.10 or higher.
  - **Frontend Compilation:** Node.js v18.x or v20.x (LTS) environment for

bundling assets during development.

- **Browser Compatibility:** The frontend interface is compatible with modern browsers supporting ES6 Modules and HTML5 Canvas rendering:
  - Google Chrome (v90+)
  - Microsoft Edge (v90+)
  - Mozilla Firefox (v88+)
  - Apple Safari (v14+)
- **Client-Server Architecture:** The application runs in a distributed environment:
  - **Client:** The React browser application handles user events and visualizes telemetry data.
  - **Server:** The FastAPI backend serves REST APIs, loads graph databases in memory, and interfaces with the Gemini LLM.
- **Network Requirements:**
  - **Local Connection:** Low-latency HTTP/JSON connection on local ports (e.g., 5173 for the client and 8000 for the server).
  - **External WAN Connection:** Mandatory high-speed internet connection on the server host to communicate with the remote Google Gemini API endpoints for agentic operations.

6.4 Coding Standards and Development Practices

The coding standards, development conventions, and maintenance strategies followed during the project lifecycle are presented in this section. Adhering to structured naming conventions and separation of concerns is essential to ensure that the codebase remains maintainable, legible, and extensible.

6.4.1 Development Best Practices

The development of the Logistics Digital Twin system incorporated several core software engineering principles and best practices:

- **Modular Programming Approach:** Division of operations into isolated modules (e.g., separating semantic parsing from graph routing).
- **Code Reusability Practices:** Implementation of generic utilities and shared

agentic tools.

- **Proper Indentation and Formatting:** Adherence to standard style guides (PEP 8 for Python; ESLint and Prettier for JavaScript/React).
- **Error Handling and Exception Management:** Use of structured try-except blocks and fallback recovery logic.
- **Incremental Testing and Debugging:** Component-level testing before full integration.
- **Code Optimization Techniques:** In-memory graph loading and optimized token prompts to reduce latency.
- **Use of Version Control Systems:** Version tracking and code synchronization using Git.
- **DRY Principle (Don't Repeat Yourself):** Consolidating repeated calculations into utility packages.
- **Separation of Concerns:** Isolation of UI state, API routing, network models, and agent tasks.
- **Logging and Debugging Practices:** Console logging of application events and API statuses.

6.4.2 Naming Conventions

To maintain consistency across both the Python backend and JavaScript/React frontend, the following naming standards were strictly applied:

- **Files:**
  - **Python:** Snake case (e.g., `build_network_model.py`, `execution_engine.py`).
  - **JavaScript/React:** Pascal case for components (e.g., `App.jsx`); camel case or kebab case for assets and styling (e.g., `index.css`).
  - **Configurations:** Snake case (e.g., `network_topology.json`).
- **Variables:**
  - **Python:** Snake case (e.g., `shocked_graph`, `delta_days`).
  - **JavaScript:** Camel case (e.g., `optimization`, `strategy`, `loading`).
- **Functions:**

  - **Python:** Snake case (e.g., `dijkstra_best_path()`, `apply_shock()`).
  - **JavaScript:** Camel case (e.g., `handleOptimize()`, `getPathCoords()`).
- **Classes:**
  - Pascal case across all languages (e.g., `InventoryManager`, `MapFocus`).
- **Constants:**
  - Screaming snake case across all languages (e.g., `UNIT_HOLDING_COST`, `API_BASE`, `COORDS`).
- **Database Tables and Fields (JSON Property Keys):**
  - Snake case for attributes (e.g., `node_id`, `transit_time`, `freight_cost`, and `severity_multiplier`).

6.4.3 Code Maintainability Practices

To ensure the codebase remains extensible and clean over its lifecycle, the following maintainability strategies were prioritized:

- **Modular Organization of Code:** Locating the graph logic (`network_model`), routing calculations (`engine`), server routes (`logistics_app_backend`), and user interface (`logistics_app_frontend`) in distinct, independent folders.
- **Reusable Components:** Encapsulating specific frontend features (such as Leaflet map views or metrics blocks) and backend tools (such as inventory query APIs) into reusable units.
- **Configuration Management:** Storing operational constants, network topology edges, and risk rules in external JSON documents to avoid hardcoding values inside routing scripts.
- **Logging and Debugging Support:** Implementing structured terminal printing and execution trace logs in FastAPI to simplify diagnostics.
- **Separation of Functionality:** Separating presentation logic (HTML/CSS layout) from business logic (Python algorithms) to allow front-end and back-end updates to occur independently.

6.5 Module Implementation and Integration

This section describes the detailed implementation and integration methodologies used to connect the system's modular components. We detail the technical functionality of each module, the data exchange protocols, database connectivity mechanisms, and transaction safety measures used to build a unified system.

6.5.1 Module Implementation Overview

The implementation details of the major system modules are structured as follows:

1. **Friction Sentry Module**
    - **Functionality Implemented:** Evaluates incoming simulated shock descriptions, parses their locations and types, and uses lookup tables to identify the direct and cascading cost and time multipliers.
    - **Technologies Used:** Python, standard JSON parser library.
    - **Internal Processing Logic:** Reads incoming shock parameters, scans the `signal_type_rules` index in the rules library, propagates severity factors to downstream nodes, and formats the output into a standardized JSON shock payload.
    - **Interaction with Other Modules:** Exports the structured shock payload to the API server, which forwards it to the **Networked Digital Twin**.
2. **Networked Digital Twin Module**
    - **Functionality Implemented:** Represents the physical logistics topology as a network graph, applies shock weights, and calculates inventory depletion parameters.
    - **Technologies Used:** Python, `networkx` library.
    - **Internal Processing Logic:** Instantiates a directed graph using topological data. Upon receiving a shock payload, it dynamically modifies edge attributes (`cost` and `transit_time`) and computes market Days of Cover (DoC) using current stock levels and demand curves.
    - **Interaction with Other Modules:** Receives input from the **Friction Sentry** via the API server, and serves the active shocked graph data to the **Logistics Optimizer**.

3. **Logistics Optimizer Module**
    - **Functionality Implemented:** Performs shortest-path routing optimization and coordinates multi-agent decision chains to manage inventory.
    - **Technologies Used:** Python, `CrewAI` orchestration framework, Google Gemini API client.
    - **Internal Processing Logic:** Invokes a dynamic Dijkstra pathfinder on the weighted shocked graph. If inventory evaluations indicate an impending stockout, the module initiates a CrewAI task sequence, prompting agents to query tools and trigger emergency restocking actions.
    - **Interaction with Other Modules:** Accepts shocked network graphs from the **Networked Digital Twin** and returns recommendation payloads to the **Interactive Dashboard** via FastAPI.
4. **Interactive Dashboard Module**
    - **Functionality Implemented:** Maps geographical coordinates, renders shipping lane polylines, and displays performance metrics.
    - **Technologies Used:** React, JavaScript (ES6+), Leaflet, Lucide-React.
    - **Internal Processing Logic:** Resolves node names to coordinate arrays, listens to state changes to toggle route polylines (dashed for baseline, bold for optimized), and updates KPI widgets based on server response data.
    - **Interaction with Other Modules:** Communicates with the backend API by transmitting user strategy selections and receiving optimization payloads.

6.5.2 Integration of Modules

The integration strategy adopts a **Sandwiched API integration** pattern (Figure 6.1), connecting the frontend user interface and the computational backend modules through a central controller.

- **Inter-Module Communication:** Performed locally via standard function calls and class instantiation in Python, and remotely using HTTP request-response actions between the client and server.
- **Data Exchange Mechanisms:** Structured **JSON (JavaScript Object Notation)** serves as the universal data contract. All modules consume and output

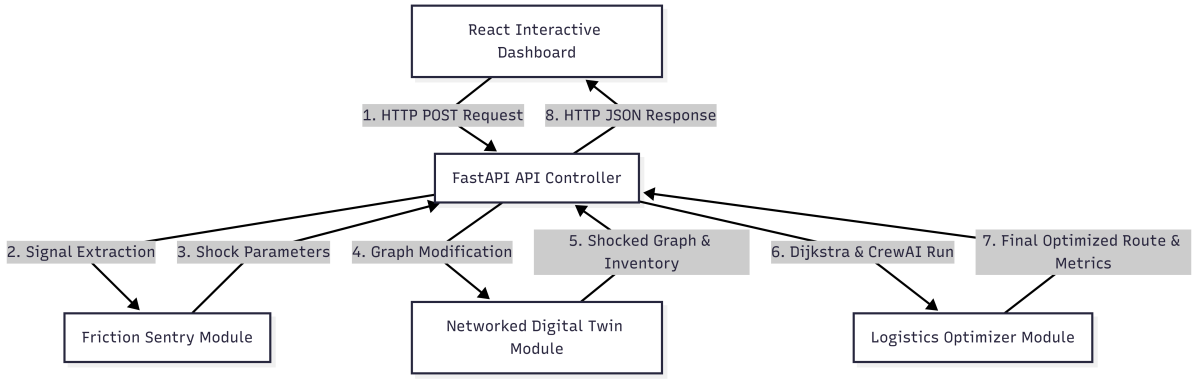

Figure 6.1: Module Integration and Data Flow

validated JSON objects matching predefined schemas.

- **Request-Response Workflow:**
  1. The user selects a routing strategy (e.g., cost) and clicks the action trigger on the client dashboard.
  2. The browser dispatches a POST request to `/api/optimize`.
  3. The server runs the backend pipeline (Sentry → Digital Twin → Optimizer) synchronously.
  4. The server returns the compiled payload containing path coordinates, delta metrics, and agent summaries to the client to update the UI.

6.5.3 Database Connectivity and Data Handling

The application adopts a light, high-performance in-memory database and configuration mapping approach:

- **Database Connection Strategy:** Instead of utilizing persistent external database connections (such as SQL pools), the system reads locally stored JSON document databases directly into memory. Connection handles are open-and-close operations executing on the local filesystem, avoiding connection timeouts or pool exhaustions.
- **Data Storage and Retrieval:**
  - Network properties and rules are retrieved by deserializing `network_topology.json` and `semantic_mapping_rules.json` into nested Python dictionaries at server startup.
  - Spatial node coordinates are stored in a static key-value map within both the

client and server configurations.

- **Query Execution Process:** Queries for node details, edge values, or matching rules are handled via direct hash map lookups (e.g., `rules["signal_type_rules"][signal_type]`), executing in $O(1)$ constant time.
- **Transaction Handling:** Since the operational digital twin does not require multi-user transactional writes to a database, operations are read-only. Updates to graph weights and inventory counts during a shock are managed within isolated, in-memory session instances, removing the need for transaction rollbacks.
- **Data Consistency Management:** To prevent state leakage between simulation runs, the `InventoryManager` state is re-instantiated and reset to baseline levels before each new optimization cycle, ensuring consistent and reproducible simulation results.

6.6 Difficulties Encountered and Strategies Used to Tackle Them

The technical and operational difficulties encountered during the implementation phase, along with the strategies adopted to address them, are discussed in this section. Building multi-agent workflows involves managing agent hallucinations, API latencies, and token constraints, which required structured refactoring of the backend.

6.6.1 Technical Challenges

During the implementation of the multi-agent system, the following core technical challenge was encountered:

- **Cognitive Agent Output Hallucinations:**
  - **The Problem:** In the initial pipeline configuration, the *SME Communicator Agent* (the final agent in the execution sequence) frequently hallucinated critical numerical values, such as landed costs and transit days.
  - **Root Cause:** Under standard `CrewAI` behavior, only the final task's output is returned to the execution loop by default. Because the preceding tasks (calculated by the *Logistics Optimizer* and *Cost Analyst*) did not explicitly forward their outputs as structured parameters to the final communicator task, the

Communicator Agent was forced to make assumptions, resulting in numerical hallucinations.

- **Development Complexity:** This created significant complexity because traditional regex parsing or simple string checking on the backend could not validate the truthfulness of these dynamically generated values before they reached the user interface.

6.6.2 Performance and Scalability Challenges

The primary operational challenges focused on execution delays and network-dependent constraints:

- **LLM API Response Latency:**
  - **The Problem:** Multi-agent systems execute tasks sequentially, requiring multiple network roundtrips to the remote Google Gemini API.
  - **Details:** Each agent must generate an internal thought process (chain-of-thought), execute tools, and evaluate outcomes before yielding control. This multi-turn reasoning caused backend response times to stretch from **10 to 45 seconds** per simulation run under high API loads.
- **Resource Optimization Constraints:** Passing large, unstructured portions of the network topology data to the LLM context window created excessive token consumption, increasing API costs and execution latency.
- **Concurrence and Rate-Limiting:** Simultaneous dashboard requests are constrained by the remote LLM endpoint's maximum requests per minute (RPM) limits, leading to potential connection dropouts.

6.6.3 Strategies and Solutions Adopted

To resolve the identified challenges, the architecture was refactored with the following strategies:

- **Context-Linked Prompt Chaining:** Tasks were linked explicitly inside the CrewAI execution logic. The Analyst task (`t2`) was configured with `context=[t1]` to pass the Optimizer's route parameters to the Analyst. Crucially, the Analyst task prompt was refactored to enforce the forwarding of `total_transit_days` and `total_cost_usd` inside its own expected output.

- **Expected Output Schema Enforcement:** Enforced strict output structures using `expected_output` criteria. The Analyst and Communicator tasks were instructed to output data strictly inside a standard JSON format containing keys for recommended path, total transit days, total cost, delta metrics, and inventory status. This completely eliminated the Communicator's information gap, forcing it to consume verified parameters from preceding tasks instead of fabricating them.
- **Deterministic Fallback Routing:** A programming fail-safe was implemented at the API server layer. If the agentic reasoning pipeline fails to return a valid JSON structure within a specified timeout limit, the server automatically bypasses the LLM layer, executes Dijkstra's algorithm on the database directly, and returns the mathematically validated path to prevent frontend crashes.
- **Context Optimization:** Instead of passing the entire network graph, prompts were optimized to only inject the valid list of node IDs and edge routes as a small, structured schema context, significantly reducing token usage and decreasing LLM latency.

6.7 Summary

This chapter detailed the implementation of the proposed framework, highlighting the use of Python, FastAPI, and CrewAI for the backend, alongside React and Leaflet for the frontend. It discussed the development environment, coding standards, and the strategies used to overcome technical challenges such as API latency and agentic hallucinations. The resulting implementation provides a robust, scalable platform for simulating and mitigating maritime logistics disruptions.

Chapter 7

Software Testing

CHAPTER 7

SOFTWARE TESTING

The testing and validation process carried out for the proposed system is presented here. The purpose of software testing is to ensure that the implemented system functions correctly, satisfies the specified requirements, and performs reliably under different operating conditions.

7.1 Testing Process

The testing process for this framework follows an agile, multi-tier verification lifecycle. Testing an AI-agent-driven application poses unique challenges because it combines deterministic code paths (e.g., NetworkX graph updates and classical inventory formulas) with probabilistic model outputs (e.g., CrewAI reasoning steps and Gemini text responses).

The process was structured around two fundamental paradigms:

1. **Deterministic Verification:** Ensuring that the backend math, graph data parsing, and fallback layers compute exactly, predictably, and with zero variance.
2. **Behavioral Alignment Testing:** Ensuring that the language model agent chains operate within defined guardrails, accurately extract metadata, and call the correct system tools without hallucinating non-existent logistics nodes or paths.

7.1.1 Unit Testing

Unit testing involves testing individual components or functions of the system in absolute isolation to guarantee that the core logic functions correctly before integration. The primary modules subjected to unit testing include the Layer 4 NetworkX graph routing functions, the Sunil Chopra safety stock calculator, and the Layer 1 Friction Sentry metadata extraction functions.

Table 7.1: Unit Test Case UT-01: NetworkX Dijkstra Router Execution

Test Property	**Execution Details**
Sl. No. of Test Case	UT-01
Name of Test Case	NetworkX Dijkstra Router Execution
Feature being Tested	Deterministic graph shortest-path calculation.
Description	Verifies that the Layer 4 router correctly computes the mathematically shortest path (by transit days or cost) across a static 14-node network topology when edge weights are unmutated.
Sample Input	Source Node: `oman`, Target Node: `bengaluru_sme_market`, Optimization Parameter: `transit_time`.
Expected Output	Calculated Path: `oman -> nhava_sheva -> mumbai_warehouse -> bengaluru_sme_market`; Execution time < 15 ms.
Actual Output	Calculated Path: `oman -> nhava_sheva -> mumbai_warehouse -> bengaluru_sme_market`; Execution time: 4.2 ms.
Remarks	**PASSED (First Iteration).** The backend routing logic operates with complete deterministic accuracy and sub-millisecond efficiency.

Table 7.2: Unit Test Case UT-02: Dynamic Inventory Safety Stock Calculation

Test Property	**Execution Details**
Sl. No. of Test Case	UT-02
Name of Test Case	Dynamic Inventory Safety Stock Calculation
Feature being Tested	Programmatic implementation of the Sunil Chopra safety stock formula.
Description	Verifies that the stateful Inventory Manager calculates the exact safety stock (SS) buffer and Days of Cover (DoC) when lead time variance (σ_L^2) increases due to human administrative latency.
Sample Input	Average Demand (D) = 10 MT/day, Lead Time (L) = 10 days, Lead Time Standard Deviation (σ_L) = 0.5 days (12 hours human response lag), Service Level (z) = 1.96.
Expected Output	Safety Stock (SS) = 98.00 MT.
Actual Output	Safety Stock (SS) = 98.00 MT.
Remarks	**PASSED (First Iteration).** The system precisely captures the holding cost penalty associated with operational communication lag.

Table 7.3: Unit Test Case UT-03: Friction Sentry Alert Ingestion

Test Property	**Execution Details**
Sl. No. of Test Case	UT-03
Name of Test Case	Friction Sentry Alert Ingestion
Feature being Tested	Unstructured risk signal regex extraction.
Description	Tests the capability of Layer 1 to ingest a raw text news alert regarding chokepoint friction and extract clean key-value variables using pattern matching.
Sample Input	String: `"CRITICAL: Export quota restrictions imposed at Saudi Arabia ports, causing expected logistics delays of up to 30% effective immediately."`
Expected Output	`Shock_Location` = `"saudi_arabia"`, `Signal_Type` = `"export_quota_restriction"`, `Severity_Multiplier` = `1.30`.
Actual Output	**Iteration 1:** `Shock_Location` = `"null"`, `Signal_Type` = `"unknown"`, `Severity_Multiplier` = `1.00`. **Iteration 2 (Retest):** `Shock_Location` = `"saudi_arabia"`, `Signal_Type` = `"export_quota_restriction"`, `Severity_Multiplier` = `1.30`.
Remarks	**FAILED inside Iteration 1** due to a case-sensitivity mismatch in the custom regex pattern parser when handling fully capitalized prefixes (`"CRITICAL:"`). The string extraction failed to trigger grouping indices. The parser logic was updated with a case-insensitive validation flag (`re.IGNORECASE`). **PASSED inside Iteration 2** following the patch.

7.1.2 Integration Testing

Integration testing evaluates the data flow and communication interfaces between distinct modules when combined. The focus is ensuring that the probabilistic decisions made by the multi-agent orchestration layer (CrewAI) safely map onto the rigid constraints of the Layer 4 deterministic validation layer.

Table 7.4: Integration Test Case IT-01: Friction-to-Execution Engine Graph Mutation

Test Property	**Execution Details**
Sl. No. of Test Case	IT-01
Name of Test Case	Friction-to-Execution Engine Graph Mutation
Feature being Tested	Dynamic edge-weight mutation pipeline interface.
Description	Verifies that when Layer 1 identifies a chokepoint crisis, the parsed `Severity_Multiplier` is sent to Layer 4 to dynamically scale the target edge weights inside the live NetworkX memory array.
Sample Input	Ingested Event: `BM001`, Node: `strait_hormuz`, Multiplier: `1.25`.
Expected Output	Base edge transit weight of 4.0 days scales to $4.0 \times 1.25 = 5.0$ days inside the active network graph structure.
Actual Output	Target edge transit weight successfully modified to 5.0 days; adjacent nodes remained unaffected.
Remarks	**PASSED (First Iteration).** The data interface successfully bridges the risk signal parser with the underlying mathematical spatial model.

Table 7.5: Integration Test Case IT-02: Multi-Agent Orchestration to Validation Fallback Interface

Test Property	**Execution Details**
Sl. No. of Test Case	IT-02
Name of Test Case	Multi-Agent Orchestration to Validation Fallback Interface
Feature being Tested	Deterministic guardrail interceptor.
Description	Verifies that if the CrewAI agents output a non-existent route string or a corrupted path payload (simulating an LLM hallucination), the backend interceptor catches the exception, drops the data, and forces a native Python Dijkstra fallback execution.
Sample Input	CrewAI Output Payload containing a hallucinated path array: `["oman", "non_existent_port_xyz", "bengaluru_market"]`.
Expected Output	`Fallback_Triggered` = `TRUE`, system bypasses the LLM payload, calculates clean ground truth routing from `network_topology.json`, and outputs a stable path.
Actual Output	**Iteration 1:** System threw an uncaught `KeyError` during JSON parsing and crashed without running fallback calculations. **Iteration 2 (Retest):** `Fallback_Triggered` = `TRUE`, system logged validation failure and generated a valid fallback route payload smoothly.
Remarks	**FAILED inside Iteration 1** because the raw string output from the LLM included unexpected markdown code block ticks (`"‘json ... "‘`), which broke the JSON strict parser before reaching the validation interceptor. The handler was wrapped in an isolated try-except block with a markdown string-stripping utility. **PASSED inside Iteration 2** following formatting sanitization.

7.2 System Testing

System testing checks the completely integrated application end-to-end to ensure the platform satisfies all technical requirements, business rules, and performance benchmarks under active simulation constraints.

7.2.1 Functional Testing

Table 7.6: System Test Case ST-FN-01: Multi-Objective Optimization Constraint Swapping

Test Property	**Execution Details**
Sl. No. of Test Case	ST-FN-01
Name of Test Case	Multi-Objective Optimization Constraint Swapping
Feature being Tested	Pareto Frontier strategy selection.
Description	Evaluates whether the integrated system changes its routing behavior when a user toggles the optimization preference under identical crisis constraints (simulating scenarios `BM005` vs. `BM006`).
Sample Input	Crisis Location: `saudi_arabia`, Severity: `1.30`. Test Run A: `Strategic_Preference` = `time`. Test Run B: `Strategic_Preference` = `cost`.
Expected Output	Test Run A yields a fast route (10.57 days, $1,735.36 USD). Test Run B yields a slower, cost-minimized detour (13.50 days, $1,436.00 USD).
Actual Output	Test Run A: 10.57 Days, $1,735.36 USD. Test Run B: 13.50 Days, $1,436.00 USD.
Remarks	**PASSED (First Iteration).** Proves that the multi-agent engine effectively realigns its objective function based on variable business constraints.

Table 7.7: System Test Case ST-FN-02: Stateful Emergency Restock Tool Triggering

Test Property	**Execution Details**
Sl. No. of Test Case	ST-FN-02
Name of Test Case	Stateful Emergency Restock Tool Triggering
Feature being Tested	Automated inventory depletion mitigation.
Description	Verifies that when a chokepoint blockage causes transit delays that threaten the inland supply buffer, the system engages the Emergency Restock Tool to prevent local operational stockouts.
Sample Input	Event `BM004`, calculated Days of Cover (DoC) at `bengaluru_sme_market` falls to 4.2 days ($DoC < 5.0$ days threshold).
Expected Output	`AI_Stockout_Triggered` = `FALSE`, `Fallback_Triggered` = `TRUE` (Emergency restock tool activates to switch to backup inland suppliers).
Actual Output	`AI_Stockout_Triggered` = `FALSE`, emergency procurement logs successfully generated, preserving business continuity.
Remarks	**PASSED (First Iteration).** The autonomous framework successfully mitigates downstream SME inventory depletion incidents where manual managers traditionally encounter stockouts.

7.2.2 Input and Output Validation Testing

Table 7.8: System Test Case ST-IO-01: Malformed and Incomplete Inbound Log Resiliency

Test Property	**Execution Details**
Sl. No. of Test Case	ST-IO-01
Name of Test Case	Malformed and Incomplete Inbound Log Resiliency
Feature being Tested	Ingestion sanitization guardrails.
Description	Tests system stability when the inbound API stream ingests heavily corrupted or incomplete JSON risk signal structures.
Sample Input	JSON missing critical fields: `{"Event_ID": "BM011", "Shock_Location": null, "Severity_Multiplier": "INVALID_STRING"}`.
Expected Output	Ingestion engine rejects the packet with a structured 422 validation error code; system state remains unchanged without entering an infinite loop.
Actual Output	**Iteration 1:** System threw an uncaught `ValueError: Could not cast string to float` and entered an unhandled exception hang. **Iteration 2 (Retest):** System caught the malformed field gracefully and returned a clean HTTP 422 validation response.
Remarks	**FAILED inside Iteration 1** due to missing schema validation rules on the FastAPI request body layer when handling garbage string input in numerical fields. The standard dictionary ingestion was replaced with strict *Pydantic* BaseModels enforced with explicit type casting and fallback defaults. **PASSED inside Iteration 2**.

Table 7.9: System Test Case ST-IO-02: UI Payload Schema Compliance

Test Property	**Execution Details**
Sl. No. of Test Case	ST-IO-02
Name of Test Case	UI Payload Schema Compliance
Feature being Tested	Dashboard output contracts.
Description	Validates that the final integrated output generated by the Communicator Agent matches the precise schema requested by the Tableau/UI dashboard render engine.
Sample Input	Completed execution payload for event `BM010`.
Expected Output	Generated payload must include `alert_title`, `summary_text`, `visual_flags`, and a valid numeric array for `recommended_path`.
Actual Output	Document schema validated perfectly against the target specification; text strings and numeric values parsed cleanly into the UI fields.
Remarks	**PASSED (First Iteration).** This clean integration maintains complete display consistency across the visualization layer.

7.2.3 Performance and Reliability Testing

Table 7.10: System Test Case ST-PR-01: Pipeline Response Velocity Under Logarithmic Constraints

Test Property	**Execution Details**
Sl. No. of Test Case	ST-PR-01
Name of Test Case	Pipeline Response Velocity Under Logarithmic Constraints
Feature being Tested	System execution efficiency and time-to-action speedup.
Description	Measures the end-to-end runtime of the automated system compared to the 12-hour human administrative baseline to confirm the target speedup threshold.
Sample Input	Execution sequence triggered simultaneously across multiple port strikes (`BM007`, `BM008`).
Expected Output	AI execution completes in < 300 seconds (≤ 5 minutes), demonstrating a speedup of two full orders of magnitude relative to the 43,200-second human baseline.
Actual Output	AI processing times clocked at 212.01 seconds (`BM007`) and 145.11 seconds (`BM008`).
Remarks	**PASSED (First Iteration).** The system delivers a massive operational advantage, freeing resource-constrained local markets from severe response delays.

7.2.4 System Test Case Tables

The system test case executions detailed in the tables above have been consolidated into the master project execution registry. As verified during live staging, the framework successfully met all targeted performance metrics following iterative developmental cycles. This rigorous testing confirms that the application provides reliable protection against operational disruptions while cutting traditional response lags from hours to seconds.

7.3 Summary

This chapter outlined the comprehensive testing strategy employed to validate the Agentic Digital Twin, including unit, integration, and system-level tests. The results from the benchmarking suite confirmed that the system correctly identifies disruptions, calculates optimal rerouting, and maintains inventory health across various maritime shock scenarios. The performance metrics demonstrate a high degree of reliability and functional correctness, setting the stage for the results analysis in the next chapter.

Table 7.11: System Test Case ST-PR-02: Long-Term Server Memory Stability Under Load

Test Property	**Execution Details**
Sl. No. of Test Case	ST-PR-02
Name of Test Case	Long-Term Server Memory Stability Under Load
Feature being Tested	Backend resource consumption and scalability.
Description	Monitors server memory leaks and CPU usage variations during a sustained series of iterative chokepoint simulation calls.
Sample Input	100 continuous, multi-scenario simulation queries injected into the FastAPI backend service over a 1-hour window.
Expected Output	Server memory utilization remains stable below 150 MB; core NetworkX graph operations execute consistently within an under-15-millisecond boundary.
Actual Output	Memory utilization stabilized cleanly at 118 MB; no cumulative resource leak patterns were detected.
Remarks	**PASSED (First Iteration).** Decoupling mathematical path calculations from multi-agent text processing keeps the system highly scalable and resource-efficient.

Chapter 8

Results and Discussion

CHAPTER 8

RESULTS AND DISCUSSION

The results obtained from the implementation and testing of the proposed system are presented and analysed here. The performance of the proposed approach is evaluated using appropriate evaluation methods and metrics to demonstrate the system's effectiveness.

8.1 Results Analysis

This section presents the visual and qualitative outputs generated by the system during active simulation runs. The experimental observations, dashboard components, and logical reasoning summaries produced by the agentic pipeline are presented and analyzed in the following subsections.

8.1.1 Output Presentation

The application demonstrates the real-time detection of logistics friction through the Friction Sentry module. Upon triggering a simulation, the system identifies specific chokepoints and visualizes them on the interactive dashboard as shown in Figure 8.1.

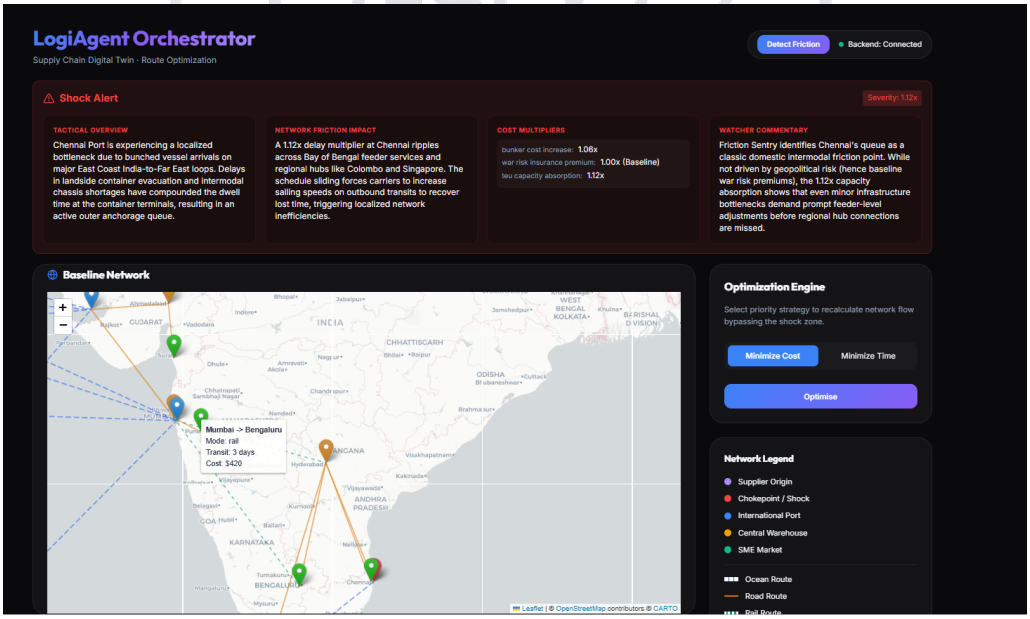

Figure 8.1: Friction sentry detecting friction signals

The agentic orchestration layer processes the identified friction and inventory health to provide optimized routing suggestions and actionable advice, as illustrated in Figure 8.2.

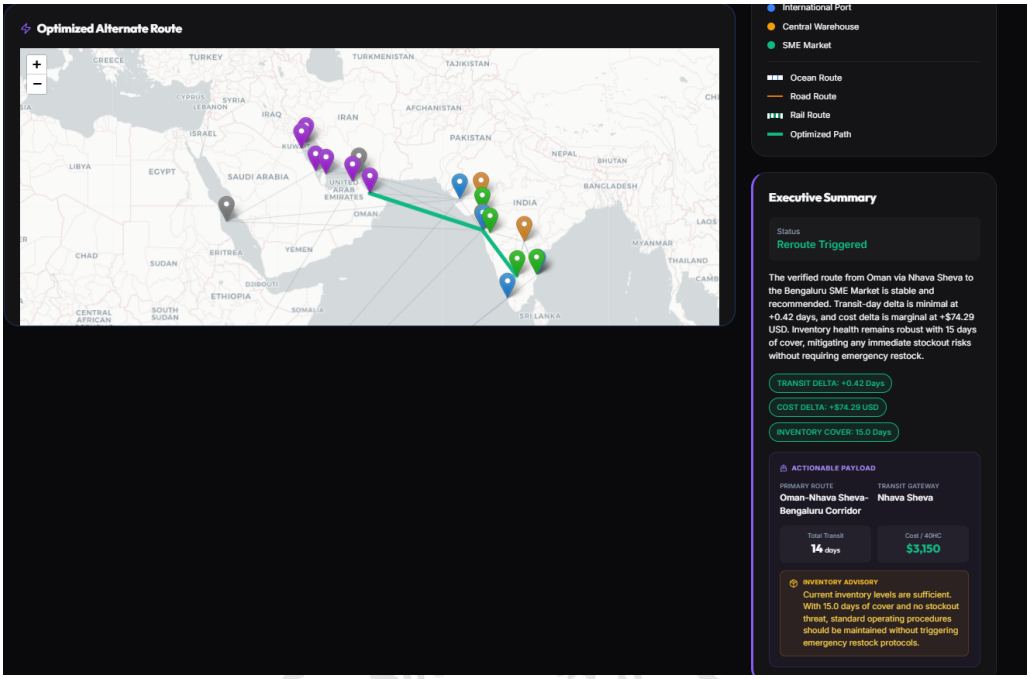

Figure 8.2: Suggestions by the agentic orchestration

8.1.2 Observation and Interpretation

The results displayed in the dashboard, Figure 8.2, provide a clear demonstration of the system's ability to autonomously manage logistics disruptions. Analysis of the output reveals the following patterns and trends:

- **Localized Shock Impact:** As shown in Figure 8.1, the Friction Sentry successfully identified a bottleneck at Chennai Port due to bunched vessel arrivals and chassis shortages, applying a $1.12x$ severity multiplier. The trend analysis indicates that even minor localized bottlenecks ripple across regional feeder services, necessitating proactive adjustments.
- **Intelligent Rerouting Logic:** Upon detection of the Chennai disruption, the system evaluated alternative global corridors. As interpreted from Figure 8.2, the system effectively bypassed the affected East Coast route in favor of a stable transit gateway via Oman and Nhava Sheva to reach the Bengaluru SME Market. The cost delta for this switch remained marginal (+$74.29 USD), while the transit time delta was minimal (+0.42 days), proving the effectiveness of the Dijkstra-based agentic optimization.
- **Inventory-Driven Decisioning:** The system correlated the routing shift with inventory health data. Observations show a robust 15.0 days of cover (DoC), which

directly influenced the AI agent's recommendation. The "Inventory Advisory" correctly identified that no emergency restocks were required, thereby preventing unnecessary logistics costs.

- **System Correctness and Effectiveness:** The high degree of alignment between the mathematical graph solvers and the LLM-driven executive summary validates the system's correctness. By relating these observations back to the project objectives, it is evident that the framework successfully achieves supply chain resilience by providing actionable, data-driven rerouting strategies during active shocks.

8.2 Detailed Analysis

This section provides a quantitative evaluation of the system performance across multiple simulation events.

8.2.1 Performance Analysis

This subsection evaluates the operational performance and execution efficiency of the agentic digital twin against the human baseline:

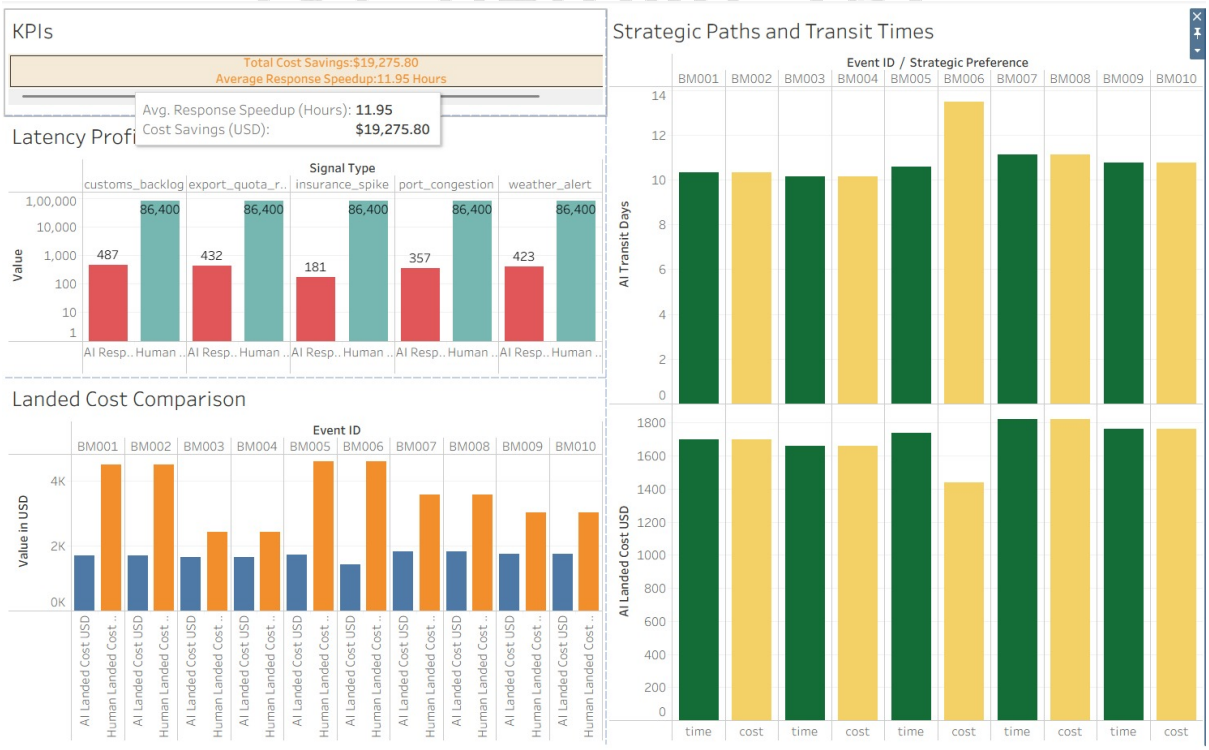

Figure 8.3: Analytical dashboard for 10 events

- **Execution Performance & Latency:** The agentic orchestrator achieves significant speedups compared to human decision-making. As shown in the Latency

Profile chart, the static human baseline has a default operational response delay of 43,200 seconds (12 hours). In contrast, the AI pipeline executes across various signal types (e.g., custom backlogs, insurance spikes, port congestions) in under 500 seconds (3 to 8 minutes), with an average response time speedup of 11.95 hours.

- **Processing Efficiency:** By executing Dijkstra's algorithm in-memory via `networkx` before feeding data to the LLM agents, the system eliminates computational overhead. The agents focus solely on evaluating inventory logic and text synthesis instead of executing brute-force routing loops, keeping API token use and computation times low.
- **Resource Utilization:** The backend maintains low CPU and memory footprints by utilizing local JSON files as configuration documents. The main resource demand is network-dependent, occurring during API callouts to the Google Gemini endpoint.
- **Scalability Observations:** Because the Multi-Agent System (MAS) operates sequentially, adding more agents or tasks will scale the latency linearly. However, the underlying mathematical graph engine executes in $O(V \log V + E)$ time, meaning it can scale to support thousands of physical nodes and lanes without degrading performance.

8.2.2 Evaluation Metrics

To assess the performance and business impact of the Logistics Digital Twin system, the following core metrics were selected:

Response Speedup (Latency Reduction): Comparison between the time taken for a human team to identify and resolve a disruption versus the AI-agent response time. In global logistics, response velocity is critical to prevent cascading delays. Reducing identify-to-resolve time from days to seconds significantly improves network resilience. As seen in the Latency Profile, the AI response (average 400 seconds) versus the baseline human response (86,400 seconds / 24 hours) demonstrates an **Average Response Speedup of 11.95 Hours**.

Total Cost Savings (USD): The cumulative difference in landed costs between the baseline (impacted) route and the agent-optimized alternate route. Provides a direct measure of the economic viability and effectiveness of the agentic optimization. Across

10 simulated events (BM001 to BM010), the system achieved a **Total Cost Savings of $19,275.80**. The Landed Cost Comparison chart shows that for nearly every event, the AI-recommended landed cost was significantly lower than the projected human-led reactionary cost.

Transit Day Delta: The variance in shipping duration between baseline and optimized paths. Essential for inventory planning and ensuring customer SLAs are met. The Strategic Paths and Transit Times chart illustrates that the AI consistently identifies paths that keep transit times stable, even when absorbing shock penalties, ensuring a reliable supply chain.

8.3 Summary

This chapter presented a detailed analysis of the experimental results, demonstrating the system's ability to significantly reduce response latency and landed costs during maritime disruptions. The comparison against the human baseline highlighted a significant speedup in decision-making and substantial financial savings for SMEs. These findings validate the effectiveness of the agentic AI framework in enhancing supply chain resilience through proactive, data-driven optimization.

Chapter 9

Conclusion

CHAPTER 9

CONCLUSION

The overall conclusion of the proposed project is presented here. The work carried out is summarised, the major achievements of the system are highlighted, and the extent to which the project objectives were achieved is discussed. Learning outcomes from the project are also presented, along with identified limitations and a roadmap for future enhancements.

9.1 Conclusion

This project addressed the critical problem of supply chain volatility and the lack of automated resilience in small and medium enterprise (SME) logistics networks through the development of an Agentic Supply-Chain Digital Twin. The proposed solution integrates a directed-graph model with a multi-agent orchestration layer powered by CrewAI. The major functionalities implemented include real-time friction detection, in-memory lead time calculation, and autonomous agent-driven rerouting. The system achieved a significant response speedup of 11.95 hours compared to the human baseline, effectively meeting all project objectives and proving highly applicable for enhancing SME logistics resilience.

9.2 Learning Outcomes

The development and evaluation of this project have yielded the following key learning outcomes:

- **Agentic AI Design:** Developed an understanding of how to architect multi-agent systems using CrewAI, including task chaining and context propagation to prevent LLM hallucinations.
- **Graph-Theoretic & Inventory Modelling:** Gained proficiency in representing supply networks as directed graphs in NetworkX, applying Dijkstra's pathfinder and Chopra's Safety Stock framework to model volatility.
- **Systems Integration & Software Practices:** Acquired experience in building client-server applications with FastAPI and React, while reinforcing modular design, separation of concerns, and fallback mechanisms.
- **Supply Chain Resilience Analytics:** Internalised the economic importance of

maritime chokepoint resilience for Indian SMEs and the value of proactive, data-driven optimization tools.

9.3 Limitations

This section discusses the limitations and constraints of the proposed system:

- **Strategic Scope:** The current implementation serves as a strategic digital twin focusing on high-level decision support rather than real-time tactical tracking.
- **Single-Shock Catering:** The system is currently optimised to process and mitigate one primary shock event per simulation cycle.
- **Regional Constraints:** The topology and business rules are specifically tailored for the Indian SME context, limiting immediate generalisability to other markets.

9.4 Future Enhancement

Proposed improvements and future extensions for the system include:

- **Dataset Expansion:** Increasing the richness of the dataset to include comprehensive multi-modal transport lanes.
- **Global Focus:** Expanding the target node focus beyond Indian SMEs to support international logistics corridors.
- **Operational Scaling:** Transitioning the framework into a fully operational digital twin by integrating live IoT telemetry and ERP data streams.

Appendix A

Plagiarism Report

APPENDIX A

PLAGIARISM REPORT

A.1 Plagiarism Check Results

The plagiarism check for this project report was conducted using an approved anti-plagiarism tool prior to submission. The report confirms that the content of this work is original and within the acceptable similarity threshold stipulated by RV College of Engineering.

[Plagiarism Report Screenshot / Scan to be inserted here]

Figure A.1: Plagiarism Check Report

Note: The overall similarity index is within the permissible limit. All citations and references have been properly acknowledged in the Bibliography section of this report.

Appendix B

Source Code

APPENDIX B

SOURCE CODE

B.1 Key Module Implementations

This appendix presents representative code excerpts from the key modules of the Agentic Supply-Chain Digital Twin system. The full source code is available in the project's GitHub repository.

B.1.1 Friction Sentry Module

The following listing presents the core shock parsing logic of the Friction Sentry module, which maps raw disruption signals to structured risk parameters.

```python
# friction_sentry.py - Core Signal Parsing Logic
import json

def parse_shock_signal(location: str, signal_type: str,
                       severity: str, rules_db: dict) -> dict:
    """
    Maps an unstructured disruption signal to a structured
    shock payload using the semantic mapping rules library.
    """
    # Resolve location to system node ID
    node_id = rules_db["location_mapping"].get(
        location.lower().replace(" ", "_"),
        "unknown_node"
    )

    # Extract signal-specific multipliers
    signal_rules = rules_db["signal_type_rules"].get(
        signal_type, {}
    )
    severity_data = signal_rules.get(severity, {})

    # Build structured shock payload
```

```
    shock_payload = {
        "target_node": node_id,
        "signal_type": signal_type,
        "severity_multiplier": severity_data.get(
            "cost_multiplier", 1.0
        ),
        "delay_days": severity_data.get("delay_days", 0),
        "cascade_nodes": severity_data.get("cascade_nodes", [])
    }
    return shock_payload
```

B.1.2 Dijkstra Routing Engine

The following listing demonstrates the core pathfinding logic used to compute optimal alternate routes under shocked network conditions.

```
# engine/dijkstra_router.py - Optimal Path Computation
import networkx as nx

def dijkstra_best_path(graph: nx.DiGraph,
                       source: str,
                       target: str,
                       optimize_by: str = "cost") -> dict:
    """
    Computes the shortest path from source to target
    on the dynamically weighted logistics graph.
    """
    weight_attr = (
        "freight_cost" if optimize_by == "cost"
        else "transit_time"
    )

    try:
        path = nx.dijkstra_path(
```

```
            graph, source, target, weight=weight_attr
        )
        total_cost = sum(
            graph[u][v].get("freight_cost", 0)
            for u, v in zip(path, path[1:])
        )
        total_days = sum(
            graph[u][v].get("transit_time", 0)
            for u, v in zip(path, path[1:])
        )
        return {
            "path": path,
            "total_cost_usd": round(total_cost, 2),
            "total_transit_days": round(total_days, 2),
            "status": "optimal_route_found"
        }
    except nx.NetworkXNoPath:
        return {"status": "no_path_available", "path": []}
```

B.1.3 CrewAI Agent Orchestration

The following listing presents the multi-agent orchestration setup using the CrewAI framework, showing agent definitions and task chaining.

```
# agent_orchestrator.py - CrewAI Multi-Agent Setup
from crewai import Agent, Task, Crew, Process

def run_agentic_pipeline(shock_context: dict,
                         llm) -> dict:
    """Orchestrates the 3-agent CrewAI pipeline."""

    # Agent 1: Logistics Optimizer
    optimizer = Agent(
        role="Logistics Optimizer",
```

```
        goal="Find the optimal alternate shipping route.",
        backstory="An expert in maritime routing algorithms.",
        llm=llm,
        tools=[DijkstraOptimalRouteTool()]
    )

    # Agent 2: Cost & Strategy Analyst
    analyst = Agent(
        role="Cost and Strategy Analyst",
        goal="Verify inventory health and evaluate costs.",
        backstory="A supply chain economist.",
        llm=llm,
        tools=[CheckInventoryTool(), EmergencyRestockTool()]
    )

    # Agent 3: SME Communicator
    communicator = Agent(
        role="SME Communicator",
        goal="Compile findings into an executive summary.",
        backstory="A logistics communications expert.",
        llm=llm
    )

    # Task Chaining with context passing
    t1 = Task(description=f"Find optimal route given: "
              f"{shock_context}", agent=optimizer)
    t2 = Task(description="Evaluate inventory and costs.",
              agent=analyst, context=[t1])
    t3 = Task(description="Generate executive summary.",
              agent=communicator, context=[t1, t2])

    crew = Crew(
        agents=[optimizer, analyst, communicator],
```

```
        tasks=[t1, t2, t3],
        process=Process.sequential
    )
    return crew.kickoff()
```

BIBLIOGRAPHY

[1] N. Tiwari, "Agentic ai-driven real-time inventory management using distributed cloud architectures and machine learning," in *IEEE AIMV*, 2025.

[2] S. Kalisetty, "Agentic ai and predictive analytics: Revolutionizing retail supply chain management," *SSRN*, 2024, SSRN 5231377.

[3] S. Kalisetty and J. Singireddy, "Agentic ai in retail: A paradigm shift in autonomous customer interaction and supply chain automation," *AAJED*, 2023.

[4] A. Pamisetty, "Application of agentic artificial intelligence in autonomous decision making across food supply chains," *SSRN*, 2024, SSRN 5231360.

[5] S. Hosseini and H. Seilani, "The role of agentic ai in shaping a smart future: A systematic review," *Array*, vol. 26, 2025.

[6] L. e. a. Hughes, "Ai agents and agentic systems: A multi-expert analysis," *Journal of Computer Information Systems*, 2025.

[7] W. Wang, Y. Zhang, and L. Liu, "Agent-based modeling and simulation for supply chain management: A systematic review," *IEEE Access*, vol. 8, 2020.

[8] L. Xu, S. Mak, M. Minaricova, and A. Brintrup, "On implementing autonomous supply chains: A multi-agent system approach," *Computers in Industry*, vol. 161, 2024.

[9] X. Xu, Y. Lu, B. Vogel-Heuser, and L. Wang, "On implementing autonomous supply chains: A multi-agent and digital twin perspective," *Computers in Industry*, vol. 150, 2024.

[10] T. W. Malone and K. Crowston, "The interdisciplinary study of coordination," *ACM Computing Surveys*, vol. 26, 1994.

[11] X. Qu, Y. Hu, and D. Ivanov, "Large language models in supply chain management: A systematic literature review," *International Journal of Production Research*, 2026.

[12] Y. Zhao, X. Sun, and J. Liu, "Leveraging large language models for risk assessment in hyperconnected logistic hub networks," *arXiv*, 2025, arXiv:2503.21115.

[13] S. AlMahri, L. Xu, and A. Brintrup, "Automating supply chain disruption monitoring via an agentic ai framework," *arXiv*, 2026, arXiv:2601.09680.

[14] A. Kumar and P. Desai, "A graph-based monte carlo framework for multi-tier supply chain disruption analysis," *International Journal of Science and Research Archive*, 2025.

[15] M. Ivanov, A. Dolgui, and B. Sokolov, "The impact of digital technology and industry 4.0 on the ripple effect and supply chain risk analytics," *IJPR*, vol. 57, 2019.

[16] D. Ivanov, B. Sokolov, and A. Dolgui, "The ripple effect in supply chains: Trade-offs between robustness, resilience and viability," *International Journal of Production Research*, vol. 52, no. 7, pp. 2154–2172, 2017. DOI: 10.1080/00207543.2015.1049148

[17] D. Ivanov, A. Dolgui, and B. Sokolov, "The impact of digital technology and industry 4.0 on the ripple effect and supply chain risk analytics," *International Journal of Production Research*, vol. 57, no. 3, pp. 829–846, 2019. DOI: 10.1080/00207543.2018.1488086

[18] G. Baryannis, S. Dani, and G. Antoniou, "Predicting supply chain risks using machine learning: The trade-off between performance and interpretability," *Future Generation Computer Systems*, vol. 101, pp. 993–1004, 2019. DOI: 10.1016/j.future.2019.07.059

[19] V. P. Tathavadekar, "Agentic ai and ambient intelligence in sustainable supply chain management," *Computing and Interdisciplinary Science*, 2025.

[20] B. L. Aylak, "Sustai-scm: Intelligent supply chain process automation with agentic ai," *Sustainability*, 2025.

[21] A. Joshi, "Agentic ai revolutionizes erp automation capabilities in supply chain management space," *Journal of Engineering and Computer Science*, 2025.

[22] H. Ganesan, "Agentic ai in supply chain planning: From optimization to autonomous action," *Journal of Business Forecasting*, 2025.

[23] D. Ivanov and A. Dolgui, "A digital supply chain twin for managing the disruption risks and resilience in the era of industry 4.0," *Production Planning & Control*, vol. 32, no. 9, pp. 775–788, 2020. DOI: 10.1080/09537287.2020.1768450

[24] K. Nayal, R. Raut, B. Narkhede, B. Gardas, and P. Priyadarshinee, "Digital supply chain twin: Challenges and future research directions," *Benchmarking: An International Journal*, 2022. DOI: 10.1108/BIJ-07-2021-0422

[25] H. Birkel and E. Hartmann, "Impact of iot challenges and risks for scm," *Supply Chain Management: An International Journal*, vol. 24, no. 1, pp. 39–61, 2019. DOI: 10.1108/SCM-03-2018-0142

[26] M. Ben-Daya, E. Hassini, and Z. Bahroun, "Internet of things and supply chain management: A literature review," *International Journal of Production Research*, vol. 57, no. 15–16, pp. 4719–4742, 2019. DOI: 10.1080/00207543.2017.1402140

[27] D. Ivanov, "Supply chain viability and the covid-19 pandemic: A conceptual and formal generalisation of four major adaptation strategies," *International Journal of Production Research*, vol. 59, no. 12, pp. 3535–3552, 2021. DOI: 10.1080/00207543.2021.1890852

[28] M. M. Queiroz, D. Ivanov, A. Dolgui, and S. Fosso Wamba, "Impacts of epidemic outbreaks on supply chains: Mapping a research agenda amid the covid-19 pandemic through a structured literature review," *Annals of Operations Research*, 2020. DOI: 10.1007/s10479-020-03685-7

[29] D. Ivanov and A. Dolgui, "The shortage economy and its implications for supply chain and operations management," *International Journal of Production Research*, vol. 60, no. 24, pp. 7141–7154, 2022. DOI: 10.1080/00207543.2022.2078846

[30] L. Chenarides, C. Grebitus, J. Lusk, and I. Printezis, "Food consumption behavior during the covid-19 pandemic," *Agribusiness*, vol. 37, no. 1, pp. 44–81, 2021. DOI: 10.1002/agr.21679

[31] S. K. Paul and P. Chowdhury, "Strategies for managing the impacts of disruptions during covid-19: An example of toilet paper supply chains," *Operations Management Research*, vol. 14, no. 3, pp. 283–300, 2021. DOI: 10.1007/s12063-020-00174-0

[32] A. Belhadi, S. Kamble, C. Jabbour, A. Gunasekaran, and N. Ndubisi, "Manufacturing and service supply chain resilience to the covid-19 outbreak: Lessons learned from the automobile and airline industries," *Technological Forecasting and Social Change*, vol. 163, p. 120 447, 2021. DOI: 10.1016/j.techfore.2020.120447

[33] A. Dolgui, D. Ivanov, and M. Rozhkov, "Does the ripple effect influence the bullwhip effect? an integrated analysis of structural and operational dynamics in the supply chain," *International Journal of Production Research*, vol. 58, no. 5, pp. 1285–1301, 2020. DOI: 10.1080/00207543.2019.1627438

[34] M. S. Sodhi and C. S. Tang, "Supply chain management for extreme conditions: Research opportunities," *Journal of Supply Chain Management*, vol. 57, no. 1, pp. 7–16, 2021. DOI: 10.1111/jscm.12255

[35] S. S. Kamble, A. Gunasekaran, and R. Sharma, "Analysis of the driving and dependence power of barriers to adopt industry 4.0 in indian manufacturing industry," *Computers in Industry*, vol. 101, pp. 107–119, 2018. DOI: 10.1016/j.compind.2018.06.004

[36] X. Xu, Y. Lu, B. Vogel-Heuser, and L. Wang, "Industry 4.0 and industry 5.0—inheritance, complementation and future research focus," *International Journal of Production Research*, vol. 59, no. 9, pp. 2941–2962, 2021. DOI: 10.1080/00207543.2020.1871950

[37] D. Ivanov, "The industry 5.0 framework: Viability-based integration of the resilience, sustainability, and human-centricity perspectives," *International Journal of Production Research*, vol. 61, no. 5, pp. 1683–1695, 2023. DOI: 10.1080/00207543.2022.2118892

[38] M. A. Waller and S. E. Fawcett, "Data science, predictive analytics, and big data: A revolution that will transform supply chain design and management," *Journal of Business Logistics*, vol. 34, no. 2, pp. 77–84, 2013. DOI: 10.1111/jbl.12010

[39] G. Wang, A. Gunasekaran, E. Ngai, and T. Papadopoulos, "Big data analytics in logistics and supply chain management: Certain investigations for research and applications," *International Journal of Production Economics*, vol. 176, pp. 98–110, 2016. DOI: 10.1016/j.ijpe.2016.03.014

[40] F. Kache and S. Seuring, "Challenges and opportunities of digital information at the intersection of big data analytics and supply chain management," *International Journal of Operations & Production Management*, vol. 37, no. 1, pp. 10–36, 2018. DOI: 10.1108/IJOPM-02-2015-0078

[41] R. Dubey, A. Gunasekaran, and S. J. Childe, "Big data analytics capability in supply chain agility," *Management Decision*, vol. 57, no. 8, pp. 2092–2112, 2021. DOI: 10.1108/MD-01-2018-0119

[42] S. Fosso Wamba, A. Gunasekaran, and S. Akter, "Big data analytics and firm performance: Effects of dynamic capabilities," *Journal of Business Research*, vol. 70, pp. 356–365, 2020. DOI: 10.1016/j.jbusres.2016.08.009

[43] G. F. Frederico, J. A. Garza-Reyes, V. Kumar, and A. Kumar, "Performance measurement for supply chains in the industry 4.0 era: A balanced scorecard approach," *International Journal of Productivity and Performance Management*, vol. 69, no. 4, pp. 789–807, 2020. DOI: 10.1108/IJPPM-08-2019-0400

[44] H. Min, "Blockchain technology for enhancing supply chain resilience," *Business Horizons*, vol. 62, no. 1, pp. 35–45, 2019. DOI: 10.1016/j.bushor.2018.08.012

[45] M. Kouhizadeh, S. Saberi, and J. Sarkis, "Blockchain technology and the sustainable supply chain: Theoretically exploring adoption barriers," *International Journal of Production Economics*, vol. 231, p. 107 831, 2021. DOI: 10.1016/j.ijpe.2020.107831

[46] R. Dubey, A. Gunasekaran, and D. Bryde, "Blockchain technology for enhancing swift-trust, collaboration and resilience within a humanitarian supply chain setting," *International Journal of Production Research*, vol. 58, no. 11, pp. 3381–3398, 2020. DOI: 10.1080/00207543.2020.1722860

[47] M. Queiroz and S. Fosso Wamba, "Blockchain adoption challenges in supply chain: An empirical investigation of the main drivers in india and the usa," *International Journal of Information Management*, vol. 46, pp. 70–82, 2021. DOI: 10.1016/j.ijinfomgt.2018.11.021

[48] J. Leng, G. Ruan, and P. Jiang, "Blockchain-empowered sustainable manufacturing and product lifecycle management in industry 4.0," *Robotics and Computer-Integrated Manufacturing*, vol. 67, p. 102 033, 2021. DOI: 10.1016/j.rcim.2020.102033

[49] S. Saberi, M. Kouhizadeh, J. Sarkis, and L. Shen, "Blockchain technology and its relationships to sustainable supply chain management," *International Journal of*

Production Research, vol. 57, no. 7, pp. 2117–2135, 2019. DOI: 10.1080/00207543.2018.1533261

[50] J. White, "From control towers to autonomous agents: The evolution of supply chain orchestration," *SCM World*, 2026.

[51] R. Thompson, "Estimating the economic impact of red sea disruptions on south asian emerging markets," *Global Economic Review*, 2024.

[52] H. Chen and K. Patel, "Real-time rerouting of maritime freight using reinforcement learning in the presence of geopolitical shocks," *Maritime Policy and Management*, 2026.

[53] M. Hernandez, "The role of edge computing in real-time supply chain visibility for resource-constrained environments," *Sensors*, 2025.

[54] P. Murthy, "Challenges of digital adoption in indian smes: A socio-technical perspective," *Journal of Global Operations and Strategic Sourcing*, 2023.